CS683: Advanced Computer Architecture

# Programming Assignment II

Task 1: Stride Pattern Prefetcher at L1D

Task 2: Instruction-conscious L2 replacement (I-RRIP and Dynamic I-RRIP)

Task 3: SPP with prefetch-aware Dynamic I-RRIP


| | |
|---|---|
| Haresh Gupta | 23b0931 |
| Yash Singh | 23b0934 |
| Sai Divya Teja | 24b1016 |
| Shivam Singh Yadav | 23b1007 |

# 1 Task 1: Stride Pattern Prefetcher at L1D

## 1.1 Task 1A: IP-Stride Prefetcher (`IP-stride`)

ChampSim calls the L1D prefetcher on every demand load, with the virtual address and the instruction pointer (IP) of the load. The provided `ip_stride` prefetcher works like this:

- **Tracker table:** 24 entries, fully associative, replaced by LRU. Each entry stores an IP tag (`ip & 0xFF`), the last cache-line address of that IP, the last stride (in cache lines) and the LRU rank.
- **New IP:** the LRU entry is replaced and no prefetch is issued.
- **Known IP:** the new stride is (current line − last line). A stride of 0 is ignored.
- **Trigger:** a prefetch is issued only when the new stride equals the stored stride (the same stride twice in a row).
- **Prefetch:** lines $\mathrm{cl}+s$, $\mathrm{cl}+2s$, $\mathrm{cl}+3s$ (degree 3). The loop stops when a line leaves the 4 KB page of the demand access.
- **Target level:** the L1D if less than half of the MSHRs are used, otherwise the L2.
- **Feedback:** none. The `cache_fill` hook is empty and the degree is fixed.

It captures only constant strides such as $A$, $A$+2, $A$+4. A repeating pattern such as +1, +2, +2, +2, +3 never has two equal strides in a row, so it never triggers a prefetch.

## 1.2 Task 1B: Stride Pattern Prefetcher (SPP)

SPP runs at the L1D, tracks every IP separately, never crosses a 4 KB page, supports a degree larger than 1, and checks *uniform* strides first and *non-uniform* stride patterns second. The sections below add one feature at a time. The names `SPP-V0`, `SPP-V1`, `SPP-P1`, `SPP-P2`, `SPP-P3`, `SPP-AC` and `SPP-Final` are the names used in the tables and plots.

### 1.2.1 `SPP-V0`: Basic Non-Uniform Prefetcher

- **Tracker table:** same as `IP-stride` (24 entries, 8-bit IP tag, LRU), with two extra fields per entry: a 12-bit *signature* and a *uniform counter*.
- **Signature:** the last three strides of the IP, 4 bits each:

$$\mathrm{sig}' = \big((\mathrm{sig} \ll 4) \mid (s \,\&\, \texttt{0xF})\big) \,\&\, \texttt{0xFFF}$$

- **Pattern History Table (PHT):** 4096 entries, indexed by the signature, each storing one stride. When a new stride $s$ is seen, the entry of the *old* signature is set to $s$ (“after this history, this stride came next”).
- **Uniform first:** if the new stride equals the previous stride, the IP is uniform and SPP prefetches exactly like `IP-stride`.
- **Non-uniform fallback:** otherwise the signature reads the PHT to get the next stride. The signature is then advanced with that predicted stride to predict the stride after it (chaining), until the degree is reached or the PHT entry is empty.
- **Degree:** fixed at 3. Prefetches stay in the demand page. Target level (L1D or L2) is chosen by MSHR occupancy, as in `IP-stride`.

*Example:* for the repeating strides (+1, +2, +2, +2, +3) the five 3-stride signatures are all different, $(1,2,2) \to 2$, $(2,2,2) \to 3$, $(2,2,3) \to 1$, $(2,3,1) \to 2$, $(3,1,2) \to 2$, so after one pass the PHT predicts the whole pattern.

### 1.2.2 `SPP-V1`: Fill-Based Throttling

`SPP-V0` plus a first throttling step:

- Every 1000 demand accesses, compute (prefetched lines filled into L1D) / (prefetches issued).
- Degree 3 if this ratio is ≥ 50%, degree 2 if ≥ 25%, otherwise degree 1.
- This ratio counts prefetches that arrived in the cache, not prefetches that were used later.

#### 1.2.3 `SPP-P1`: Larger Tracker Table

`SPP-V1` plus:

- Tracker table from 24 to 128 entries, IP tag from 8 to 12 bits.
- A valid bit per entry, so an empty entry cannot match an IP whose tag is 0.
- **Why:** fewer different IPs share one tracker, and trackers live long enough to collect a full stride history.

#### 1.2.4 `SPP-P2`: Clean Stride History

`SPP-P1` plus:

- **Stride range:** a stride outside $[-64, +63]$ lines restarts the history of that IP and is not stored. A page has only 64 lines, so such a stride can never give a same-page prefetch.
- **History count:** a 2-bit counter (0 to 3) per tracker. The PHT is trained and read only after 3 valid strides, so signatures padded with zeros do not pollute it.

#### 1.2.5 `SPP-P3`: PHT Confidence

`SPP-P2` plus a 2-bit saturating confidence in every PHT entry (0 means empty):

- **Entry empty:** store the stride, confidence = 1.
- **Same stride seen again:** confidence +1 (maximum 3).
- **Different stride seen:** confidence −1. At 0 the entry takes the new stride with confidence 1.
- **Prediction:** the first predicted stride needs confidence ≥ 2. Every further (chained) stride needs confidence = 3, because a chained prediction depends on the one before it.

#### 1.2.6 `SPP-AC`: Accuracy Checker

`SPP-AC` is `SPP-P3` with the fill-based throttling of `SPP-V1` replaced by a checker that measures real usefulness.

- **Prefetch filter:** 64 entries, direct-mapped by the low 6 bits of the cache-line address, each with a valid bit and a 10-bit partial tag. It remembers lines our prefetcher filled into the L1D that no demand load has used yet.
- **Fill hook:** when a prefetched line is filled (`l1d_prefetcher_cache_fill`, `prefetch=1`), insert its virtual line address into the filter and add 1 to the fill counter $F$.
- **Use hook:** a demand load that hits in the L1D (`l1d_prefetcher_operate`) and finds its line in the filter adds 1 to the useful counter $U$ and clears the entry, so each line counts once.
- **Epoch:** after $W = 256$ fills, the accuracy is $A = \lfloor 100\,U/F \rfloor$ and the degree for the next epoch is

$$d = \begin{cases} D_H & A \geq T_H \\ D_M & T_M \leq A < T_H \\ D_L & T_L \leq A < T_M \\ D_{\min} & A < T_L \end{cases}$$

  Then $F$ and $U$ are reset to 0. The degree starts at 3.
- **$D_{\min} = 1$, never 0:** epochs are counted in fills, so with degree 0 no fill would happen, the epoch would never end and the degree could never go up again.

`SPP-AC` uses 128 trackers with a 12-bit IP tag, a 64-entry filter, $W = 256$, thresholds $(T_H, T_M, T_L) = (75, 50, 25)$ and degrees $(D_H, D_M, D_L, D_{\min}) = (6, 4, 2, 1)$. Section 1.5 examines each of these values.

#### 1.2.7 `SPP-Final`: The Submitted Prefetcher

`SPP-Final` is `SPP-AC` with an **8-bit IP tag** and a **32-entry prefetch filter**. All other values are the same as in `SPP-AC`. It is the file `prefetcher/spp.l1d_pref`. Section 1.6 explains why.

#### 1.2.8 Sweep Variants

Each variant changes one value of `SPP-AC`.

| Names | What changes |
|---|---|
| `Trk64`, `Trk256` | tracker entries 64 or 256 (default 128) |
| `Tag8` | 8-bit IP tag (default 12) |
| `Flt32`, `Flt128` | prefetch filter 32 or 128 entries (default 64) |
| `Win128`, `Win512` | epoch of 128 or 512 fills (default 256) |
| `ThrStrict`, `ThrLoose` | thresholds 90/70/50 or 60/40/20 (default 75/50/25) |
| `DegHi4`, `DegHi8` | high-band degree 4 or 8 (default 6) |
| `DegMid3`, `DegMid5` | middle-band degree 3 or 5 (default 4) |
| `DegLow1`, `DegLow3` | low-band degree 1 or 3 (default 2) |
| `Fix1` to `Fix4` | throttling off, always degree 1 to 4 |

## 1.3 Metrics

$I$ is the number of simulated instructions, $C$ the number of cycles and $M$ the number of L1D load misses. Subscript $b$ is the baseline (no prefetching), $p$ is a run with a prefetcher.

$$\text{IPC} = I/C, \qquad \text{Speedup} = \text{IPC}_p/\text{IPC}_b, \qquad \text{MPKI} = 1000\,M/I,$$

$$\text{Coverage} = (M_b - M_p)/M_b \times 100\%, \qquad \text{Accuracy} = \frac{\text{useful prefetches}}{\text{prefetches issued to the next level}} \times 100\%.$$

MPKI and accuracy are read from the ChampSim statistics of the L1D. The accuracy checker inside `SPP-AC` and `SPP-Final` uses its own cheaper estimate only for throttling, never in the tables. When we compare configurations over all traces we use the geometric mean of the three speedups, and the arithmetic mean of accuracy and coverage. All runs: single core, hashed perceptron branch predictor, LRU at all levels, no other prefetcher, 25M warm-up and 25M simulated instructions, traces Trace1 to Trace3.

## 1.4 Impact of Each Feature

Table 1 shows every stage on every trace, Table 2 the averages, and Figures 1 and 2 the same data as plots. Each stage adds one feature to the stage before it, in the order of Section 1.2.

| Prefetcher | IPC | Speedup | L1D load MPKI | Accuracy (%) | Coverage (%) |
|---|---|---|---|---|---|
| *Trace1* | | | | | |
| Baseline | 0.2322 | 1.0000 | 10.68 | – | 0.00 |
| IP-stride | 0.2334 | 1.0051 | 10.12 | 52.8 | 5.24 |
| SPP-V0 | 0.2316 | 0.9975 | 9.70 | 32.4 | 9.15 |
| SPP-V1 | 0.2321 | 0.9996 | 9.82 | 35.4 | 8.03 |
| SPP-P1 | 0.2325 | 1.0012 | 9.50 | 38.2 | 11.06 |
| SPP-P2 | 0.2342 | 1.0084 | 9.83 | 56.0 | 7.96 |
| SPP-P3 | 0.2342 | 1.0086 | 9.89 | 61.4 | 7.37 |
| SPP-AC | 0.2344 | 1.0094 | 9.70 | 56.5 | 9.17 |
| SPP-Final | 0.2345 | 1.0098 | 9.68 | 57.8 | 9.37 |
| *Trace2* | | | | | |
| Baseline | 0.2511 | 1.0000 | 13.53 | – | 0.00 |
| IP-stride | 0.2512 | 1.0002 | 13.44 | 38.1 | 0.73 |
| SPP-V0 | 0.2497 | 0.9943 | 13.45 | 18.7 | 0.59 |
| SPP-V1 | 0.2499 | 0.9951 | 13.48 | 18.7 | 0.43 |
| SPP-P1 | 0.2525 | 1.0055 | 13.11 | 26.6 | 3.11 |
| SPP-P2 | 0.2535 | 1.0093 | 13.16 | 54.7 | 2.77 |
| SPP-P3 | 0.2531 | 1.0079 | 13.19 | 63.0 | 2.53 |
| SPP-AC | 0.2537 | 1.0102 | 13.07 | 59.3 | 3.45 |
| SPP-Final | 0.2545 | 1.0133 | 12.94 | 63.3 | 4.40 |
| *Trace3* | | | | | |
| Baseline | 0.3821 | 1.0000 | 3.40 | – | 0.00 |
| IP-stride | 0.3830 | 1.0024 | 3.32 | 16.1 | 2.51 |
| SPP-V0 | 0.3843 | 1.0059 | 3.07 | 18.6 | 9.83 |
| SPP-V1 | 0.3836 | 1.0042 | 3.21 | 20.5 | 5.64 |
| SPP-P1 | 0.3877 | 1.0147 | 2.91 | 31.3 | 14.48 |
| SPP-P2 | 0.3858 | 1.0097 | 3.10 | 37.3 | 8.96 |
| SPP-P3 | 0.3856 | 1.0094 | 3.12 | 40.5 | 8.25 |
| SPP-AC | 0.3869 | 1.0128 | 3.02 | 41.9 | 11.13 |
| SPP-Final | 0.3870 | 1.0128 | 3.04 | 41.0 | 10.69 |

**Table 1.** Every stage of the design on every trace (25M warm-up, 25M simulated instructions).

| Prefetcher | Geometric-mean speedup | Mean accuracy (%) | Mean coverage (%) |
|---|---|---|---|
| Baseline | 1.00000 | – | 0.00 |
| IP-stride | 1.00257 | 35.67 | 2.83 |
| SPP-V0 | 0.99923 | 23.21 | 6.52 |
| SPP-V1 | 0.99963 | 24.88 | 4.70 |
| SPP-P1 | 1.00710 | 32.04 | 9.55 |
| SPP-P2 | 1.00914 | 49.30 | 6.56 |
| SPP-P3 | 1.00863 | 55.00 | 6.05 |
| SPP-AC | 1.01079 | 52.58 | 7.92 |
| SPP-Final | 1.01200 | 54.03 | 8.15 |

**Table 2.** Average over the three traces (geometric mean for speedup, arithmetic mean for accuracy and coverage).

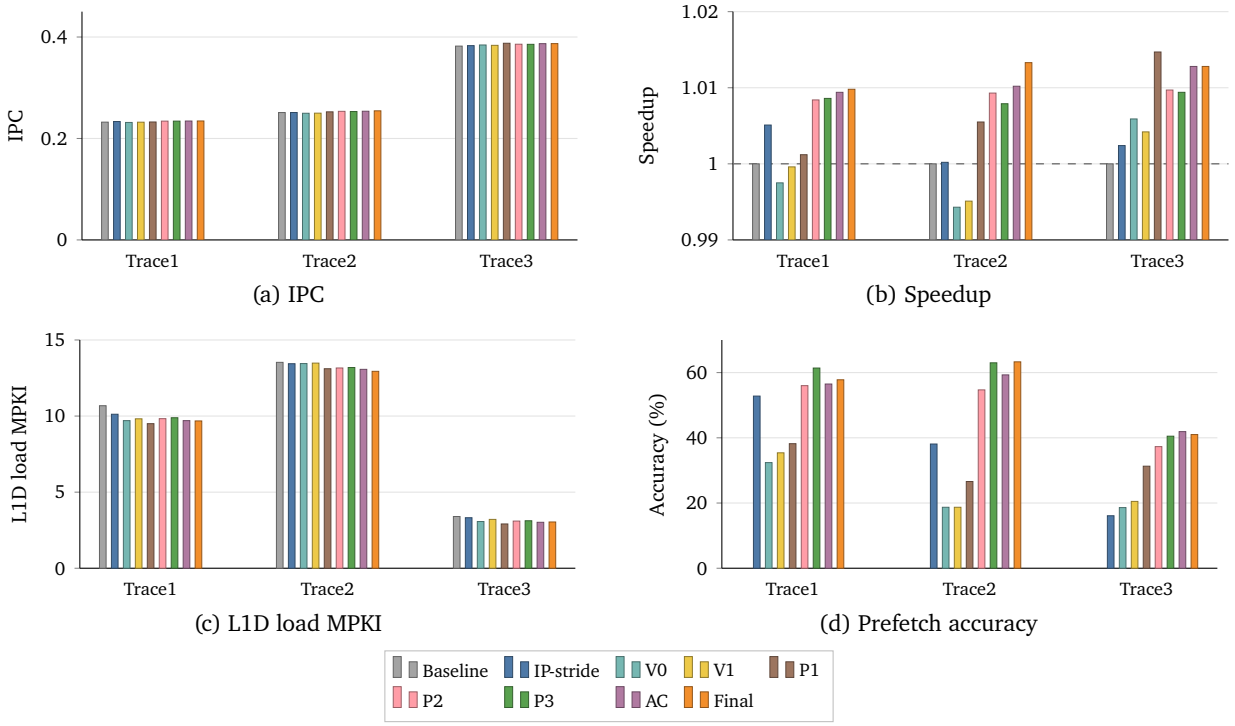



**Figure 1.** Every stage of the design. Bars in each trace group, left to right: Baseline, `IP-stride`, `SPP-V0`, `SPP-V1`, `SPP-P1`, `SPP-P2`, `SPP-P3`, `SPP-AC`, `SPP-Final`.

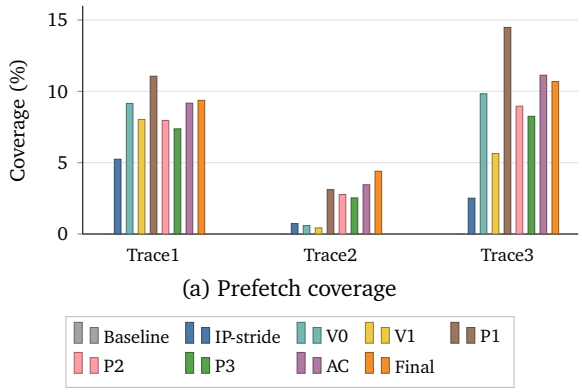



**Figure 2.** Coverage of every stage (same bar order as Figure 1). The baseline issues no prefetches, so it has no accuracy or coverage bars.

## 1.5 How Each Value Was Chosen

### 1.5.1 Method

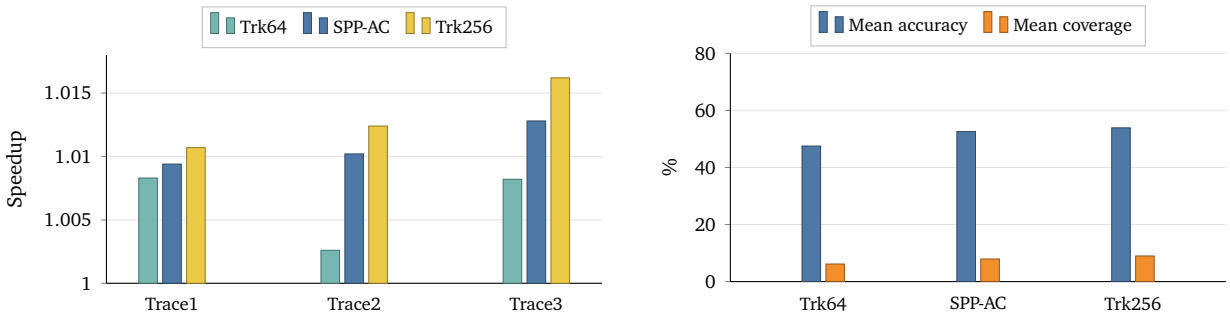
- **Starting point:** `SPP-AC`. Each sweep variant changes **one** value and keeps all others, with the same traces and instruction counts.
- **Main measure:** geometric-mean speedup over the three traces. Accuracy and coverage are used to explain the result.
- A gap below 0.05% is called a **tie**. We only have three traces and cannot tell such a small gap from trace-sample noise. The 0.05% cutoff is our own choice.
- In a tie we keep the current value. The exception is table size: equal speed with less storage wins.
- Each value was changed alone. The two changes that make `SPP-Final` were then run together (Section 1.6).
- Every subsection below gives the table and the plots of one value, then the decision with its reason. The plots use the same axis range, so they can be compared.

### 1.5.2 IP Tracker Table Entries

`Trk64`, `SPP-AC` (128) and `Trk256`.

| Variant | Value | Speedup T1 | Speedup T2 | Speedup T3 | Geomean speedup | Mean acc (%) | Mean cov (%) |
|---|---|---|---|---|---|---|---|
| `Trk64` | 64 | 1.0083 | 1.0026 | 1.0082 | 1.00636 | 47.51 | 6.16 |
| **`SPP-AC`** | 128 | 1.0094 | 1.0102 | 1.0128 | 1.01079 | 52.58 | 7.92 |
| `Trk256` | 256 | 1.0107 | 1.0124 | 1.0162 | 1.01313 | 53.89 | 8.98 |

**Table 3.** IP tracker table entries. Only this value changes; bold row = `SPP-AC`.




(a) Speedup (b) Accuracy and coverage

**Figure 3.** IP tracker table entries: speedup per trace (left) and mean accuracy and coverage (right).

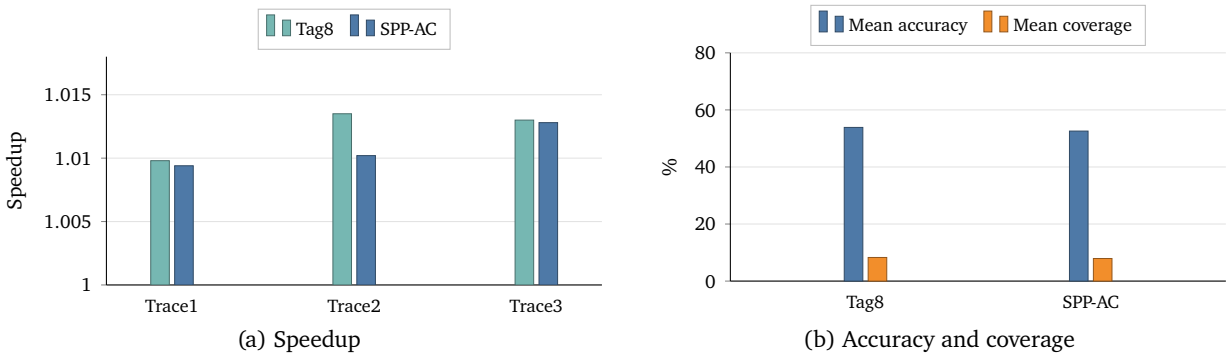
- 64 entries is clearly worse (−0.439%).
- 256 entries is better (+0.232%) and better on all three traces.
- **Cost:** 64 more entries (64 to 128) need $64 \times 84 = 5376$ bits, about 0.66 KiB, and gave about 0.67% speedup per KiB. 128 more entries (128 to 256) need $128 \times 84 = 10752$ bits, about 1.3 KiB (22% of the whole prefetcher), and give about 0.18% speedup per KiB. The second doubling gives under a third of the speedup per KiB of the first.
- **Decision: keep 128 entries.** The assignment asks for a lightweight prefetcher with maximum performance for minimal storage. The gain of the second doubling is real but small for its cost, so we accept the lower speed to stay small. If storage were not a concern, 256 would be the better choice.

### 1.5.3 IP Tag Bits

`Tag8` and `SPP-AC` (12 bits), with 128 entries.

| Variant | Value | Speedup T1 | Speedup T2 | Speedup T3 | Geomean speedup | Mean acc (%) | Mean cov (%) |
|---|---|---|---|---|---|---|---|
| `Tag8` | 8 | 1.0098 | 1.0135 | 1.0130 | 1.01206 | 53.86 | 8.28 |
| **`SPP-AC`** | 12 | 1.0094 | 1.0102 | 1.0128 | 1.01079 | 52.58 | 7.92 |

**Table 4.** IP tag bits. Only this value changes; bold row = `SPP-AC`.




(a) Speedup (b) Accuracy and coverage

**Figure 4.** IP tag bits: speedup per trace (left) and mean accuracy and coverage (right).

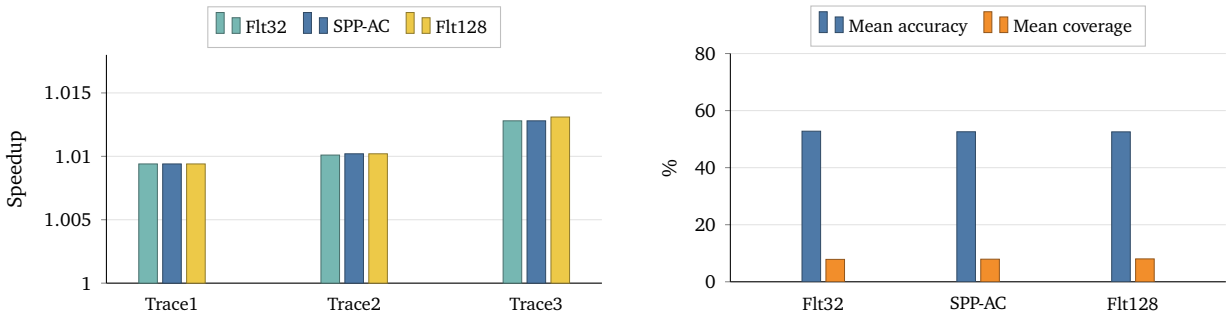
- The 8-bit tag is faster (+0.126%, a real gain) and also needs 4 bits less per tracker entry.
- **Decision: 8 bits.** Better and smaller.
- We tested only 8 and 12 bits, so 8 is not shown to be the best width. We did not investigate why the shorter tag helps.

### 1.5.4 Prefetch Filter Entries

`Flt32`, `SPP-AC` (64) and `Flt128`.

| Variant | Value | Speedup T1 | Speedup T2 | Speedup T3 | Geomean speedup | Mean acc (%) | Mean cov (%) |
|---|---|---|---|---|---|---|---|
| `Flt32` | 32 | 1.0094 | 1.0101 | 1.0128 | 1.01075 | 52.78 | 7.85 |
| **`SPP-AC`** | 64 | 1.0094 | 1.0102 | 1.0128 | 1.01079 | 52.58 | 7.92 |
| `Flt128` | 128 | 1.0094 | 1.0102 | 1.0131 | 1.01092 | 52.55 | 8.02 |

**Table 5.** Prefetch filter entries. Only this value changes; bold row = `SPP-AC`.




(a) Speedup (b) Accuracy and coverage

**Figure 5.** Prefetch filter entries: speedup per trace (left) and mean accuracy and coverage (right).

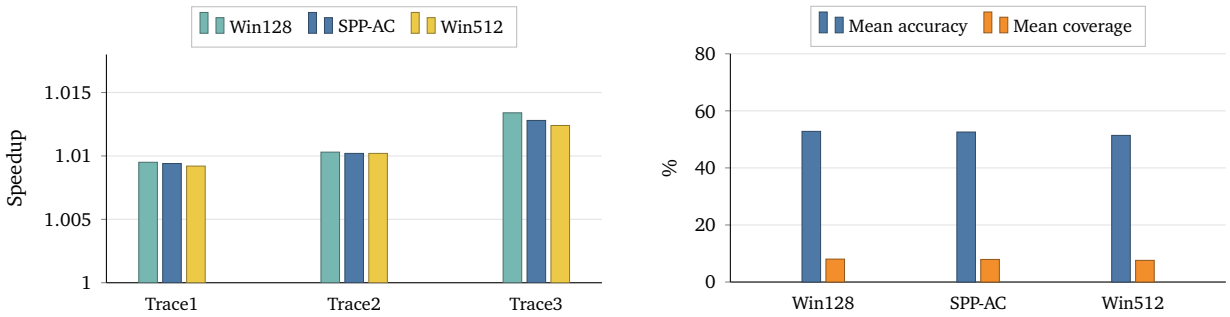
- 32, 64 and 128 entries are ties (−0.004% and +0.013%). Accuracy and coverage barely change.
- **Decision: 32 entries.** Equal speed with less storage wins.
- We did not test fewer than 32 entries.

### 1.5.5 Epoch Length

`Win128`, `SPP-AC` (256) and `Win512`.

| Variant | Value | Speedup T1 | Speedup T2 | Speedup T3 | Geomean speedup | Mean acc (%) | Mean cov (%) |
|---|---|---|---|---|---|---|---|
| `Win128` | 128 | 1.0095 | 1.0103 | 1.0134 | 1.01106 | 52.81 | 8.04 |
| **`SPP-AC`** | 256 | 1.0094 | 1.0102 | 1.0128 | 1.01079 | 52.58 | 7.92 |
| `Win512` | 512 | 1.0092 | 1.0102 | 1.0124 | 1.01058 | 51.40 | 7.60 |

**Table 6.** Epoch length (prefetch fills). Only this value changes; bold row = `SPP-AC`.




(a) Speedup (b) Accuracy and coverage

**Figure 6.** Epoch length (prefetch fills): speedup per trace (left) and mean accuracy and coverage (right).

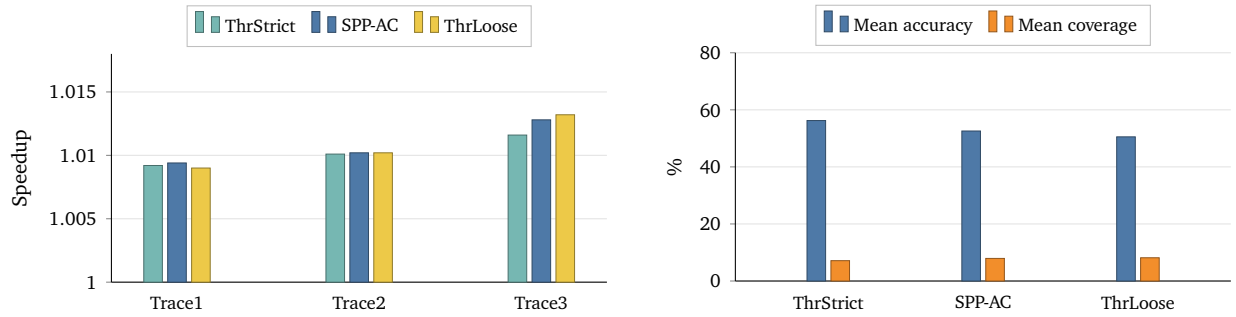
- 128, 256 and 512 fills are ties (+0.027% and −0.020%).
- **Decision: keep 256.** With 256 samples the measured accuracy has a standard error of at most $\sqrt{0.25/256} \approx 3.1$ points, much smaller than the 25-point band width, so a band is rarely chosen by noise. A shorter epoch would react faster but change the degree more often, a longer one would react slowly to program phases. The sweep shows no reason to change it.

### 1.5.6 Accuracy Thresholds

`ThrStrict` (90/70/50), `SPP-AC` (75/50/25) and `ThrLoose` (60/40/20), with the degrees 6/4/2/1.

| Variant | Value | Speedup T1 | Speedup T2 | Speedup T3 | Geomean speedup | Mean acc (%) | Mean cov (%) |
|---|---|---|---|---|---|---|---|
| `ThrStrict` | 90/70/50 | 1.0092 | 1.0101 | 1.0116 | 1.01026 | 56.24 | 7.13 |
| **`SPP-AC`** | 75/50/25 | 1.0094 | 1.0102 | 1.0128 | 1.01079 | 52.58 | 7.92 |
| `ThrLoose` | 60/40/20 | 1.0090 | 1.0102 | 1.0132 | 1.01080 | 50.53 | 8.15 |

**Table 7.** Accuracy thresholds. Only this value changes; bold row = `SPP-AC`.




(a) Speedup (b) Accuracy and coverage

**Figure 7.** Accuracy thresholds: speedup per trace (left) and mean accuracy and coverage (right).

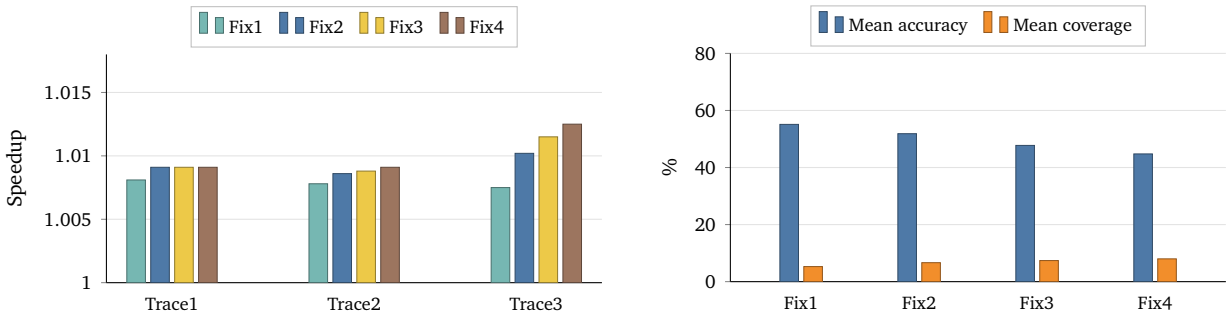
- Starting point: four degrees need four accuracy bands, so we split 0 to 100% into four equal bands, 75/50/25.
- `ThrStrict` is −0.053% against 75/50/25. Its accuracy is higher (56.2% against 52.6%) but its coverage is lower (7.13% against 7.92%): it keeps the degree low too often.
- `ThrLoose` is a tie in speedup (+0.001%) and has lower accuracy (50.5%).
- **Decision: keep 75/50/25.** Strict is slower, and loose is not faster and is less accurate.

### 1.5.7 Prefetch Degrees: Why Different Degrees for Different Accuracy

The degree is how far ahead we prefetch. Going further hides more latency when the prefetches are used, but every extra prefetch that is not used wastes bandwidth and takes a place in the L1D. So accurate streams get a high degree and inaccurate streams a low degree. The next four tables test this: fixed degrees first, then each band alone.

| Variant | Value | Speedup T1 | Speedup T2 | Speedup T3 | Geomean speedup | Mean acc (%) | Mean cov (%) |
|---|---|---|---|---|---|---|---|
| `Fix1` | 1 | 1.0081 | 1.0078 | 1.0075 | 1.00780 | 55.12 | 5.28 |
| `Fix2` | 2 | 1.0091 | 1.0086 | 1.0102 | 1.00930 | 51.86 | 6.65 |
| `Fix3` | 3 | 1.0091 | 1.0088 | 1.0115 | 1.00981 | 47.75 | 7.39 |
| `Fix4` | 4 | 1.0091 | 1.0091 | 1.0125 | 1.01024 | 44.77 | 7.99 |

**Table 8.** Fixed degree, throttling off. Only this value changes; bold row = `SPP-AC`.




(a) Speedup (b) Accuracy and coverage

**Figure 8.** Fixed degree, throttling off: speedup per trace (left) and mean accuracy and coverage (right).

### Fixed degrees (throttling off)

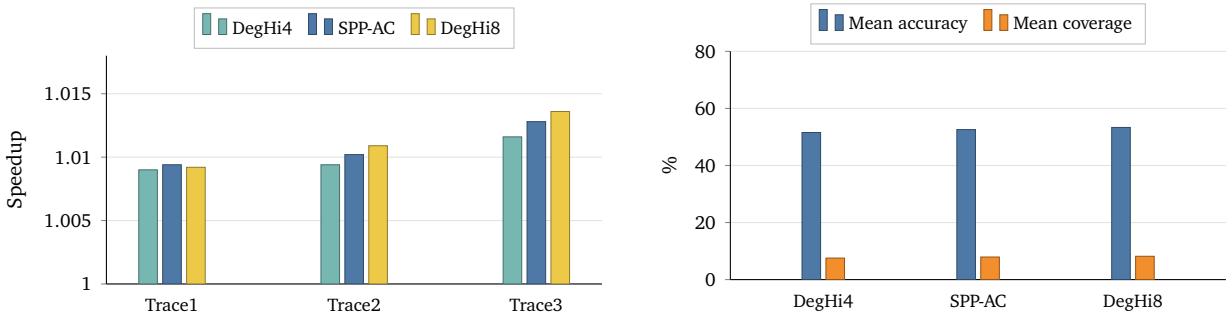
- Speedup grows with the degree, but accuracy falls (55.1%, 51.9%, 47.8%, 44.8%). A deeper degree gives speed but costs accuracy.
- `SPP-AC` (throttled, 6/4/2/1) is +0.054% against the best fixed degree (`Fix4`), with accuracy 52.6% against 44.8%. The checker gives high degrees only when prefetches are being used.

### High band (accuracy at least 75%)

`DegHi4`, `SPP-AC` (6) and `DegHi8`.

| Variant | Value | Speedup T1 | Speedup T2 | Speedup T3 | Geomean speedup | Mean acc (%) | Mean cov (%) |
|---|---|---|---|---|---|---|---|
| `DegHi4` | 4 | 1.0090 | 1.0094 | 1.0116 | 1.01004 | 51.56 | 7.55 |
| **`SPP-AC`** | 6 | 1.0094 | 1.0102 | 1.0128 | 1.01079 | 52.58 | 7.92 |
| `DegHi8` | 8 | 1.0092 | 1.0109 | 1.0136 | 1.01125 | 53.33 | 8.18 |

**Table 9.** Degree of the high band. Only this value changes; bold row = `SPP-AC`.




(a) Speedup (b) Accuracy and coverage

**Figure 9.** Degree of the high band: speedup per trace (left) and mean accuracy and coverage (right).

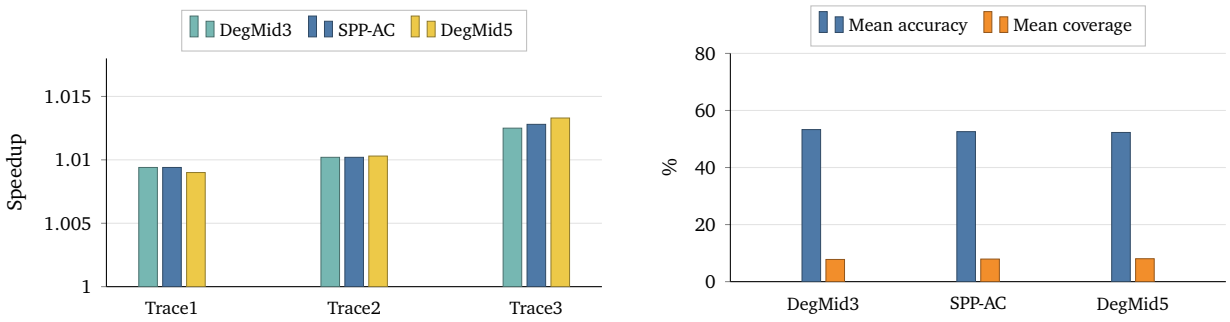
- Degree 4 is worse (−0.074%). Degree 8 is a tie (+0.045%), better on Trace2 and Trace3 but lower on Trace1.
- **Decision: keep 6.** When at least 75% of prefetches are used, a deeper degree pays off, as the fixed-degree test shows. 8 is not clearly better than 6, so we keep the smaller degree.

### Middle band (accuracy 50 to 75%)

`DegMid3`, `SPP-AC` (4) and `DegMid5`.

| Variant | Value | Speedup T1 | Speedup T2 | Speedup T3 | Geomean speedup | Mean acc (%) | Mean cov (%) |
|---|---|---|---|---|---|---|---|
| `DegMid3` | 3 | 1.0094 | 1.0102 | 1.0125 | 1.01067 | 53.30 | 7.78 |
| **`SPP-AC`** | 4 | 1.0094 | 1.0102 | 1.0128 | 1.01079 | 52.58 | 7.92 |
| `DegMid5` | 5 | 1.0090 | 1.0103 | 1.0133 | 1.01085 | 52.29 | 8.04 |

**Table 10.** Degree of the middle band. Only this value changes; bold row = `SPP-AC`.




(a) Speedup (b) Accuracy and coverage

**Figure 10.** Degree of the middle band: speedup per trace (left) and mean accuracy and coverage (right).

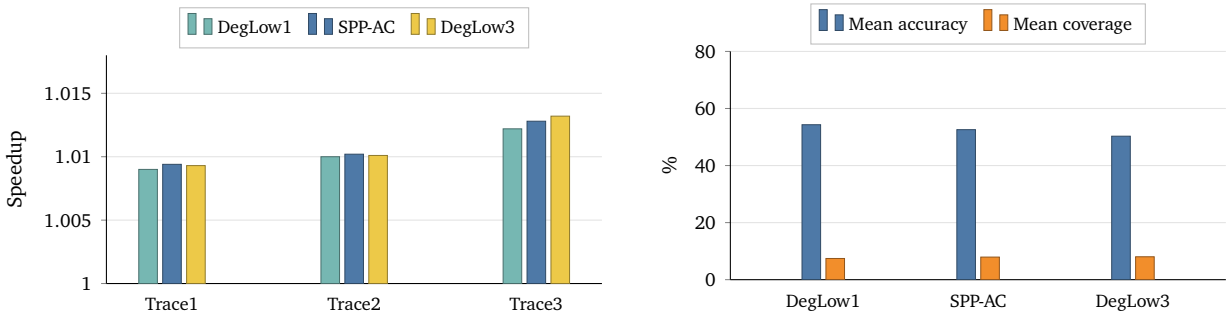
- Degree 3 (−0.011%) and degree 5 (+0.006%) are ties.
- **Decision: keep 4**, between the high-band degree 6 and the low-band degree 2. The data does not show that 4 is better than 3 or 5. It is kept as chosen.

**Low band (accuracy 25 to 50%)**

`DegLow1`, `SPP-AC` (2) and `DegLow3`.

| Variant | Value | Speedup T1 | Speedup T2 | Speedup T3 | Geomean speedup | Mean acc (%) | Mean cov (%) |
|---|---|---|---|---|---|---|---|
| `DegLow1` | 1 | 1.0090 | 1.0100 | 1.0122 | 1.01042 | 54.31 | 7.43 |
| **`SPP-AC`** | 2 | 1.0094 | 1.0102 | 1.0128 | 1.01079 | 52.58 | 7.92 |
| `DegLow3` | 3 | 1.0093 | 1.0101 | 1.0132 | 1.01086 | 50.29 | 8.03 |

**Table 11.** Degree of the low band. Only this value changes; bold row = `SPP-AC`.




(a) Speedup (b) Accuracy and coverage

**Figure 11.** Degree of the low band: speedup per trace (left) and mean accuracy and coverage (right).

- Degree 1 (−0.037%) and degree 3 (+0.007%) are ties. Degree 1 has higher accuracy (54.3%) and lower coverage (7.43%), degree 3 the opposite.
- **Decision: keep 2.** Less than half of the prefetches are used here, so we prefetch less than in the middle band. The data does not show that 2 is better than 1 or 3.

**Bottom band (accuracy below 25%)**

The degree is **1, never 0**. Without any prefetch no fill happens, the epoch never ends and the degree can never recover. All four degrees are used on every trace (Table 12), so every band is active.

| Trace | Degree 1 | Degree 2 | Degree 4 | Degree 6 | Total epochs |
|---|---|---|---|---|---|
| Trace1 | 1 | 108 | 249 | 90 | 448 |
| Trace2 | 9 | 53 | 73 | 165 | 300 |
| Trace3 | 33 | 77 | 77 | 83 | 270 |

**Table 12.** `SPP-Final`: number of 256-fill epochs that ended with each prefetch degree.

#### 1.5.8 Values Fixed by Design (not swept)

- **Stride range** −64 **to** +63**:** a page has 64 lines, so larger strides cannot give a same-page prefetch.
- **History needed** = 3 **strides:** equal to the signature length, so the PHT is used only with a full signature.
- **Minimum degree 1:** explained above.
- **Signature** = 3 **strides of 4 bits, PHT of 4096 entries:** inherited from our first version. Three strides are enough for the assignment's example pattern. We did not test other lengths. A 4-bit stride mixes strides that differ by a multiple of 16.
- **PHT confidence (2 bits, 2 to start, 3 for chained strides):** chosen by reasoning (a chained prediction depends on the one before it), not by a sweep.
- **Start degree 3:** used only until the first epoch ends. Not tested.
- **Same-page rule, MSHR rule for the target level, uniform trigger:** taken from `IP-stride`.
- **Features added in groups:** `SPP-P1` changed three things at once (128 entries, 12-bit tag, valid bit) and `SPP-P2` two (stride range, history count). They were not tested one at a time. Their reasons are in Section 1.2, and the entries and tag were then swept above.

#### 1.5.9 Summary of Decisions

| Value | Chosen | Decided by | Reason |
|---|---|---|---|
| Tracker entries | 128 | sweep + storage | 256 is faster but about 22% more storage for a small gain |
| IP tag | 8 bits | sweep | faster and smaller than 12 bits |
| Prefetch filter | 32 entries | sweep + storage | tie in speed, smaller |
| Epoch length | 256 fills | sweep + statistics | tie in speed; low sampling error |
| Thresholds | 75/50/25 | sweep | strict is slower, loose is less accurate |
| High-band degree | 6 | sweep | 4 is worse, 8 is not clearly better |
| Middle-band degree | 4 | sweep (tie) | kept as chosen |
| Low-band degree | 2 | sweep (tie) | kept as chosen |
| Bottom-band degree | 1 | design | degree 0 stops the epoch counter |
| Stride range, history needed | −64..63, 3 | design | page size, signature length |
| Signature, PHT size | 3 strides × 4 bits, 4096 | inherited | not tested |
| PHT confidence limits | 2-bit, 2 and 3 | design | chained predictions need more confidence; not tested |
| Start degree | 3 | design | used only until the first epoch ends; not tested |

**Table 13.** Every value, how it was decided and why.

## 1.6 Final Choice and Storage

`SPP-Final` combines the two changes the sweep supports, the 8-bit tag and the 32-entry filter. Each was tested alone, so we ran the combination as well (Tables 1 and 2). Its geometric-mean speedup is **1.01200** against 1.01079 for `SPP-AC` (+0.120%), with mean accuracy 54.0% against 52.6% and mean coverage 8.15% against 7.92%. Its per-trace speedups are 1.0098, 1.0133 and 1.0128 (`SPP-AC`: 1.0094, 1.0102, 1.0128).

### 1.6.1 Storage Overhead of `SPP-Final`

We assume 48-bit virtual addresses, so a cache-line address has 48 − 6 = 42 bits. Only state needed in hardware is counted (statistics counters are not).

| Field | Why this width | Bits |
|---|---|---|
| *IP tracker table, 128 entries* | | |
| Valid | entry in use | 1 |
| IP tag | `ip & 0xFF` | 8 |
| Last line address | previous cache line of this IP | 42 |
| Last stride | range $-64 \ldots + 63$ | 7 |
| Uniform flag | only compared with 0, so 1 bit is enough | 1 |
| Signature | three 4-bit strides | 12 |
| History count | 0 to 3 | 2 |
| LRU rank | rank among 128 entries | 7 |
| **Entry total** | | **80** |
| **Table total** | $128 \times 80$ | **10240** |
| *Pattern History Table, 4096 entries, indexed by signature (no tag)* | | |
| Predicted stride | range $-64 \ldots + 63$ | 7 |
| Confidence | 2-bit saturating counter (0 = empty) | 2 |
| **Entry total** | | **9** |
| **Table total** | $4096 \times 9$ | **36864** |
| *Prefetch filter, 32 entries, direct-mapped* | | |
| Valid | line prefetched and not yet used | 1 |
| Partial tag | line address bits above the 5 index bits | 10 |
| **Entry total** | | **11** |
| **Table total** | $32 \times 11$ | **352** |
| *Control registers* | | |
| Fill counter $F$ | counts to $W = 256$ | 8 |
| Useful counter $U$ | at most $256 + 32 = 288$ | 9 |
| Current degree | values 1 to 6 | 3 |
| **Total** | | **20** |
| **Total** | 10240 + 36864 + 352 + 20 | **47476 bits** = 5934.5 bytes ≈ **5.8 KiB** |

**Table 14.** Bit-level storage of `SPP-Final`. `SPP-AC` (12-bit tag, 64-entry filter) needs 48340 bits, about 5.9 KiB.

## 1.7 Final Results

IPC, speedup, L1D load MPKI, prefetch accuracy and coverage of the `IP-stride` prefetcher and our submitted prefetcher `SPP-Final`, on the three traces.

| Prefetcher | IPC | Speedup | L1D load MPKI | Accuracy (%) | Coverage (%) |
|---|---|---|---|---|---|
| *Trace1* | | | | | |
| `IP-stride` | 0.2334 | 1.0051 | 10.12 | 52.8 | 5.24 |
| `SPP-Final` | 0.2345 | 1.0098 | 9.68 | 57.8 | 9.37 |
| *Trace2* | | | | | |
| `IP-stride` | 0.2512 | 1.0002 | 13.44 | 38.1 | 0.73 |
| `SPP-Final` | 0.2545 | 1.0133 | 12.94 | 63.3 | 4.40 |
| *Trace3* | | | | | |
| `IP-stride` | 0.3830 | 1.0024 | 3.32 | 16.1 | 2.51 |
| `SPP-Final` | 0.3870 | 1.0128 | 3.04 | 41.0 | 10.69 |

**Table 15.** IPC, speedup (over no prefetching), L1D load MPKI, prefetch accuracy and coverage of `IP-stride` and the submitted prefetcher `SPP-Final`.

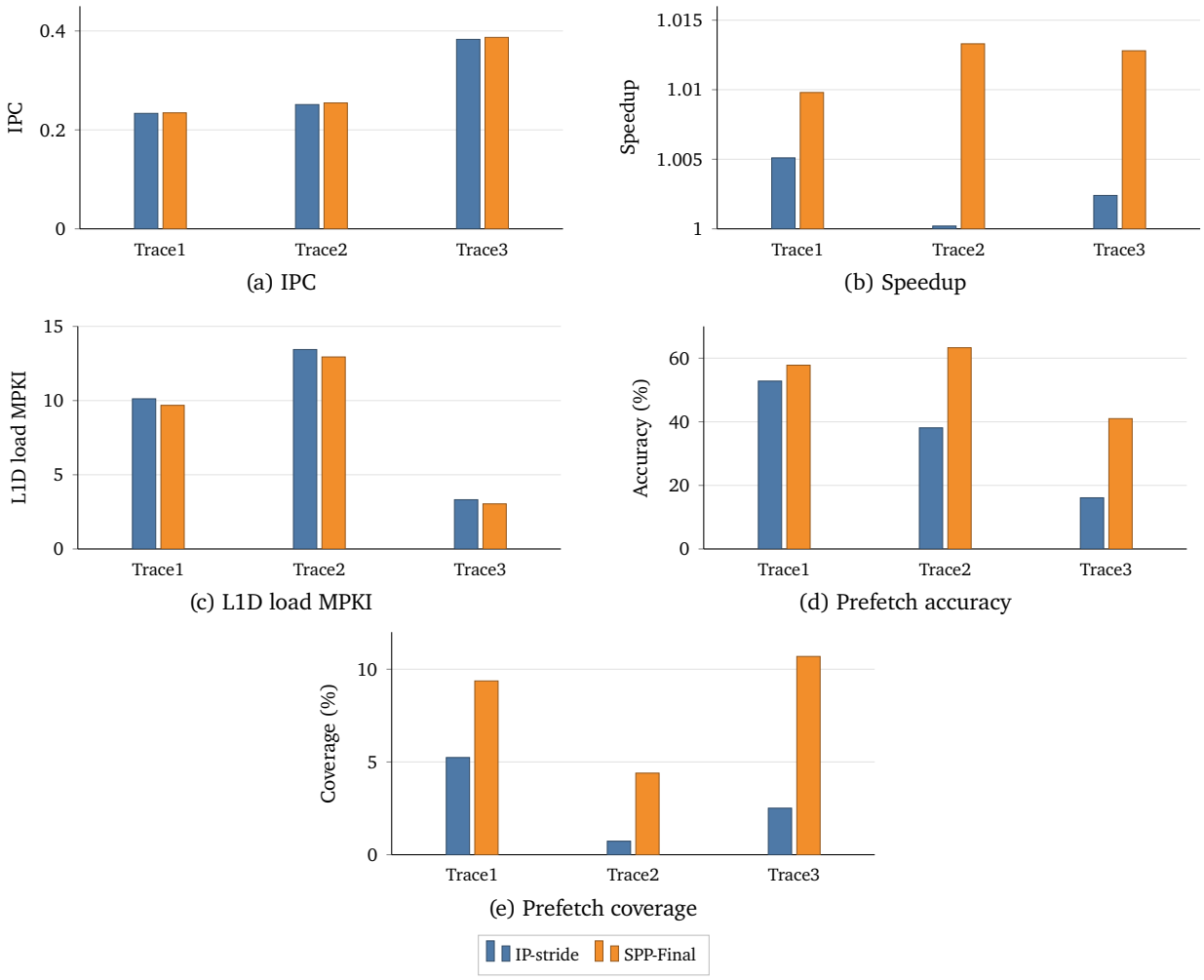



**Figure 12.** IPC, speedup, L1D load MPKI, prefetch accuracy and coverage of `IP-stride` and `SPP-Final` on the three traces.

# 2 Setup and Metrics (Tasks 2 and 3)

Tasks 2 and 3 use the same simulator and traces as Task 1. The points below list what is new or different.

- **System:** ChampSim from the PA2 repository, single core, hashed-perceptron branch predictor. L1I 32 KB 8-way, L1D 48 KB 12-way, **L2C 512 KB 8-way (1024 sets)**, LLC 2 MB 16-way. Every cache except the L2C uses LRU. There is no prefetcher unless SPP is named. 25M warm-up and 25M simulated instructions on Trace1–Trace3.
- **Baseline:** every speedup is measured against no prefetcher and LRU at the L2C (the repository default, `./build_champsim.sh no lru`). We also show SRRIP (`rrip.l2c_repl`) because I-RRIP is built on top of it.
- **Metrics:** IPC = instructions / cycles, and speedup = IPC / $\text{IPC}_{\text{baseline}}$. The MPKI values are the ChampSim ROI statistics `L2C LOAD MPKI`, `L2C INSTRUCTION LOAD MPKI` and `L2C DATA LOAD MPKI` (instruction + data = load). L1D accuracy and coverage are defined as in Task 1. When we average over the traces we use the geometric mean for IPC and speedup and the arithmetic mean for everything else.
- **Interface change (required by Task 2A):** `update_replacement_state` now takes a ninth argument, `uint8_t isInstruction`. We added it to the function-pointer type and the declarations in `inc/cache.h`, to every replacement file, and to all 11 call sites. Each call site passes the `instruction` field of the packet that causes the update (the MSHR entry on a fill, the RQ/WQ/PQ entry on a hit). A line counts as an instruction line when `isInstruction` is set and the request is a `LOAD` or `PREFETCH`. This is the same rule `fill_cache()` uses for `BLOCK::instruction`, and the same rule behind the L2 instruction MPKI statistic.
- **Two harness fixes** (neither one changes timing): (1) `PAGE_TABLE_WALKER::handle_page_fault()` is declared `uint64_t` but had no `return`. GCC ≥ 8 treats the end of such a function as unreachable, so the unmodified repository built with GCC 11 crashed after a few thousand instructions. We added `return 0;` (no caller uses the value). (2) The single-core path of `main.cc` never called `l2c_replacement_final_stats()`. We added the call so the policy can print its decision statistics.
- **Reproducibility:** every configuration in this report is one line in `scripts/experiments.sh`. Policy parameters are compile-time macros, and their defaults are the chosen values. For a sweep point, `scripts/build.sh` writes a two-line wrapper that `#define`s the macro and `#include`s the policy file.
- **SPP and compiler:** Task 3 uses the Task 1 prefetcher (`prefetcher/spp.l1d_pref`, `SPP-Final`) unchanged. Tasks 2 and 3 were compiled with GCC 11 under WSL, while Task 1 used GCC 7 in Docker, so the absolute numbers differ slightly. The baseline IPC is 0.2321 / 0.2499 / 0.3820 here and 0.2322 / 0.2511 / 0.3821 in Task 1. The speedup of SPP is 1.0083 / 1.0135 / 1.0126 here and 1.0098 / 1.0133 / 1.0128 in Task 1. All comparisons inside Tasks 2 and 3 use the same build, so they are consistent with each other.

# 3 Task 2A: Instruction-Conscious L2 Replacement (I-RRIP)

## 3.1 Why protect instruction lines in the L2?

When a load misses in the L1D and hits in the L2, it waits about 10 extra cycles, and the out-of-order core (352-entry ROB) can usually hide much of that by working on other instructions. An L1I miss is different: fetch stops until the line arrives, so the delay goes almost straight into execution time.

Our traces have large code footprints. Even with SRRIP the L2 instruction load MPKI is 10.55, 19.32 and 1.20, and on Trace2 instruction misses are 71% of all L2 load misses. Keeping instruction lines in the L2 turns these expensive front-end stalls into short L2 hits. The price is some extra data misses, which the core partly hides anyway.

## 3.2 Design

I-RRIP keeps the SRRIP hardware: a 2-bit re-reference prediction value (RRPV) per line, the victim is a line with RRPV 3, and if no line has RRPV 3 every line of the set ages by one. I-RRIP changes all three parts of the policy:

| Part | SRRIP | I-RRIP |
|---|---|---|
| Insertion | every line at RRPV 2 | instruction line at RRPV **0**, data line at RRPV **3** |
| Promotion | every hit → RRPV 0 | demand hits (instruction and data) → RRPV 0; **a writeback hit does not promote** |
| Eviction | first line (lowest way) with RRPV 3 | among lines with RRPV 3, **a data line before an instruction line** |

**Hardware cost over SRRIP:** one instruction bit per line, i.e. 8192 bits for the 512 KB L2 (ChampSim already keeps it in `BLOCK::instruction`). Everything else is just different constants and one extra comparison in the victim search.

## 3.3 How each part was chosen: ablation

Starting from SRRIP, we changed one part at a time and kept a change only if it raised the geometric-mean speedup by more than 0.05%. Smaller gaps count as ties, the same rule as in Task 1. Table 16 lists every variant and Figure 13 plots their geometric-mean speedups.

| Name | Instr. insert | Data insert | Data hit | WB hit | Victim | Speedup T1 | Speedup T2 | Speedup T3 | Geomean | L2 instr. MPKI (mean) | L2 data MPKI (mean) |
|---|---|---|---|---|---|---|---|---|---|---|---|
| SRRIP | 2 | 2 | →0 | →0 | first at 3 | 0.9996 | 1.0020 | 0.9990 | 1.0002 | 10.36 | 5.95 |
| A1 | 1 | 2 | →0 | →0 | first at 3 | 1.0004 | 1.0064 | 0.9996 | 1.0021 | 9.90 | 6.18 |
| A2 | 0 | 2 | →0 | →0 | first at 3 | 0.9996 | 1.0097 | 0.9998 | 1.0030 | 9.65 | 6.37 |
| A3 | 1 | 3 | →0 | →0 | first at 3 | 0.9974 | 1.0201 | 0.9998 | 1.0057 | 8.19 | 7.26 |
| A4 | 0 | 3 | →0 | →0 | first at 3 | 0.9970 | 1.0216 | 1.0000 | 1.0061 | 8.07 | 7.39 |
| P1 | 0 | 2 | −1 | →0 | first at 3 | 0.9992 | 1.0136 | 1.0001 | 1.0043 | 9.18 | 6.55 |
| P2 | 0 | 3 | −1 | →0 | first at 3 | 0.9964 | 1.0238 | 1.0015 | 1.0072 | 7.70 | 7.61 |
| P3 | 0 | 2 | →0 | none | first at 3 | 0.9990 | 1.0138 | 1.0004 | 1.0044 | 9.16 | 6.44 |
| P4 | 0 | 3 | →0 | none | first at 3 | 0.9974 | 1.0267 | 1.0046 | 1.0095 | 7.71 | 7.56 |
| P5 | 0 | 3 | −1 | none | first at 3 | 0.9963 | 1.0269 | 1.0059 | 1.0096 | 7.50 | 7.80 |
| E1 | 0 | 2 | →0 | →0 | data first | 0.9989 | 1.0162 | 0.9999 | 1.0050 | 9.01 | 6.57 |
| E2 | 0 | 3 | →0 | →0 | data first | 0.9961 | 1.0225 | 1.0000 | 1.0061 | 7.70 | 7.71 |
| E3 | 0 | 2 | →0 | none | data first | 0.9987 | 1.0188 | 1.0004 | 1.0059 | 8.65 | 6.62 |
| **I-RRIP** | 0 | 3 | →0 | none | data first | 0.9962 | 1.0276 | 1.0050 | 1.0095 | 7.44 | 7.91 |
| E4 | 0 | 3 | −1 | none | data first | 0.9956 | 1.0271 | 1.0061 | 1.0095 | 7.44 | 7.91 |

**Table 16.** I-RRIP ablation. The insert columns are RRPVs. "Data hit" →0 means promotion to RRPV 0, −1 means one step closer. "WB hit" is the promotion on a writeback hit. Speedup is over LRU. Highlighted: the submitted I-RRIP.

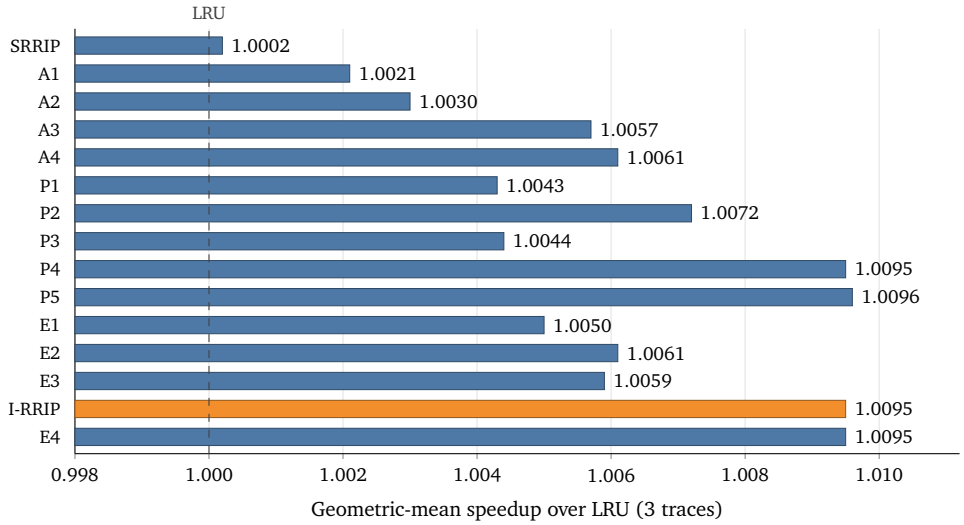



**Figure 13.** Geometric-mean speedup over LRU of every ablation variant (dashed line: LRU). Orange: the submitted I-RRIP.

**Insertion.** Inserting instruction lines at RRPV 0 instead of 2 helps (A2: +0.30%), because they survive two more aging rounds before they can be evicted. RRPV 1 protects them less and is worse (A1: +0.21%). The bigger step is inserting data at RRPV 3 (A4: +0.61%, A3: +0.57%): a new data line then becomes the first victim of its set unless it is reused before the next miss. This is a safe bet, because most L2 data lines are never reused before eviction (the L2 load hit rate of SRRIP is only 30%, 49% and 27%). Little useful data is lost and the space goes to instructions. We keep both changes: instructions at 0, data at 3.

**Promotion.** We tested two changes. Promoting a data hit by only one step (P1, P2) helps a little on its own (A4 → P2: +0.61% → +0.72%). The most important single change, however, is *not promoting on a writeback hit* (A4 → P4: +0.61% → +0.95%). Writebacks are dirty lines that the L1D has just evicted. They are 11–25% of all L2 accesses, and over 92% of them hit in the L2 because the line is still there from its fill. SRRIP moves each such line to RRPV 0, so the line the L1D just threw away gets the highest priority in the L2, ahead of every instruction line. A writeback is not a reuse by the core, so it should not change the prediction. Once writebacks stop promoting, one-step data promotion adds nothing (P4 +0.95% vs P5 +0.96%, a tie), so demand hits keep the simpler SRRIP rule (→ RRPV 0): a data line that really is reused deserves to stay.

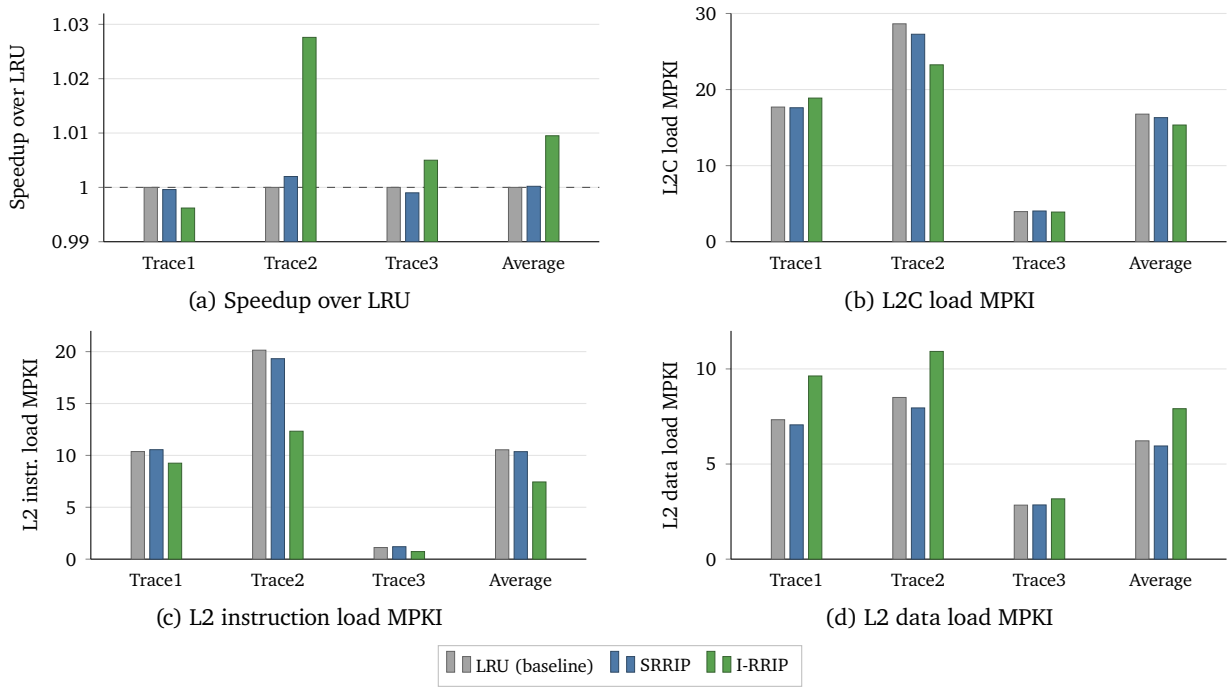
**Eviction.** Picking a data line before an instruction line among the RRPV-3 candidates matters when both kinds reach RRPV 3 together. This is common when data is inserted at 2 (A2 → E1: +0.30% → +0.50%). With data inserted at 3, most RRPV-3 lines are data lines already, and the speedup gain is a tie (P4 +0.95% vs I-RRIP +0.95%). We keep the rule anyway: it is the only part that protects an *old* instruction line, and it lowers the L2 instruction MPKI further, which is the stated goal of the task (Trace2: 12.98 → 12.33). It costs nothing beyond the instruction bit.

**Final choice.** I-RRIP ties for the highest geometric-mean speedup with P4, P5 and E4 (all within 0.01%), and together with E4 it has the lowest mean L2 instruction MPKI. E4 only adds one-step data promotion, which brings no gain, so we submit the simpler I-RRIP. The cost is visible on Trace1, the trace whose data is reused in the L2: its data MPKI rises from 7.06 to 9.63 and its speedup drops to 0.9962. Task 2B fixes this.

## 3.4 Results

| Configuration | IPC | Speedup | L2C load MPKI | L2 instr. load MPKI | L2 data load MPKI |
|---|---|---|---|---|---|
| *Trace1* | | | | | |
| LRU (baseline) | 0.2321 | 1.0000 | 17.70 | 10.37 | 7.33 |
| SRRIP | 0.2320 | 0.9996 | 17.61 | 10.55 | 7.06 |
| **I-RRIP** | 0.2312 | 0.9962 | 18.88 | 9.25 | 9.63 |
| *Trace2* | | | | | |
| LRU (baseline) | 0.2499 | 1.0000 | 28.64 | 20.14 | 8.50 |
| SRRIP | 0.2504 | 1.0020 | 27.27 | 19.32 | 7.95 |
| **I-RRIP** | 0.2568 | 1.0276 | 23.25 | 12.33 | 10.92 |
| *Trace3* | | | | | |
| LRU (baseline) | 0.3820 | 1.0000 | 3.96 | 1.12 | 2.84 |
| SRRIP | 0.3816 | 0.9990 | 4.04 | 1.20 | 2.85 |
| **I-RRIP** | 0.3839 | 1.0050 | 3.90 | 0.73 | 3.17 |
| *Average (geometric mean for IPC and speedup, arithmetic mean otherwise)* | | | | | |
| LRU (baseline) | 0.2809 | 1.0000 | 16.77 | 10.54 | 6.22 |
| SRRIP | 0.2809 | 1.0002 | 16.31 | 10.36 | 5.95 |
| **I-RRIP** | 0.2836 | 1.0095 | 15.34 | 7.44 | 7.91 |

**Table 17.** Task 2A: IPC, speedup over LRU and L2 load MPKIs (no prefetcher).




(a) Speedup over LRU

(b) L2C load MPKI

(c) L2 instruction load MPKI

(d) L2 data load MPKI

**Figure 14.** Task 2A: speedup and L2C / L2 instruction / L2 data load MPKI of LRU, SRRIP and I-RRIP.

I-RRIP lowers the mean L2 instruction load MPKI from 10.36 (SRRIP) to 7.44, i.e. 28% fewer instruction misses, while the data load MPKI rises from 5.95 to 7.91. On Trace2, where instructions cause most L2 misses, this gives a +2.76% speedup over LRU, and the total L2 load MPKI falls as well. On Trace3 the instruction misses almost disappear (1.20 → 0.73). On Trace1 the extra data misses outweigh the saved instruction misses.

# 4 Task 2B: Dynamic I-RRIP

I-RRIP helps Trace2 and Trace3 but hurts Trace1. Dynamic I-RRIP (`replacement/dyn_irrip.l2c_repl`) chooses between SRRIP and I-RRIP while the program runs.

## 4.1 Design

The policy uses **set dueling**. A few **leader sets always run SRRIP** and, in the final design, the same number of leader sets always run I-RRIP. All other sets (the followers) use whichever policy is currently winning. Leader sets are placed by complement-select: the 1024 sets are split into $N$ groups of $1024/N$ sets, and in group $k$ the set at offset $k$ is an SRRIP leader and the set at offset $1024/N - 1 - k$ is an I-RRIP leader. This spreads the leaders over the whole index range, so they do not line up with power-of-two strides of the program.

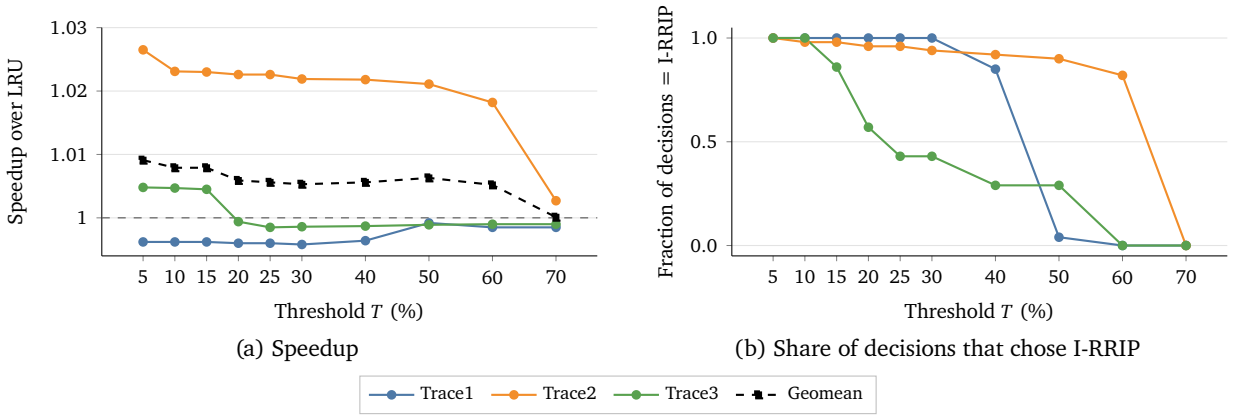
The leader sets count instruction and data (non-instruction) **demand** accesses and misses; writebacks are not counted. We implemented two decision rules (`DYN_MODE`):

- **Mode 0: the formula from the hint.** In the SRRIP leaders, ratio = (instr. accesses + instr. misses) / (total accesses + total misses). Every `DYN_EPOCH` = 1024 leader accesses, the followers use I-RRIP if ratio > $T$. The four counters are then halved so that old program phases fade out. We measure the ratio in the SRRIP sets on purpose: in the I-RRIP sets, the policy we are judging would already have removed instruction misses and biased the measurement.
- **Mode 1: miss duel (submitted).** A 10-bit saturating counter PSEL starts at 512. A demand miss in an SRRIP leader adds its weight to PSEL, and a miss in an I-RRIP leader subtracts it. A data miss weighs 1 and an instruction miss weighs $W = 2$. Every 1024 SRRIP-leader accesses, the followers use I-RRIP if PSEL $\geq$ 512, i.e. if the I-RRIP leaders produced fewer weighted misses.

## 4.2 The formula from the hint: threshold sensitivity

| Threshold | Speedup Trace1 | Speedup Trace2 | Speedup Trace3 | Geomean speedup | Fraction I-RRIP (T1/T2/T3) | Mean ratio (T1/T2/T3) |
|---|---|---|---|---|---|---|
| $T$ = 5% | 0.9962 | 1.0265 | 1.0048 | 1.0091 | 1.00 / 1.00 / 1.00 | 0.44 / 0.61 / 0.31 |
| $T$ = 10% | 0.9962 | 1.0231 | 1.0047 | 1.0079 | 1.00 / 0.98 / 1.00 | 0.44 / 0.61 / 0.31 |
| $T$ = 15% | 0.9962 | 1.0230 | 1.0045 | 1.0079 | 1.00 / 0.98 / 0.86 | 0.44 / 0.61 / 0.31 |
| $T$ = 20% | 0.9960 | 1.0226 | 0.9994 | 1.0059 | 1.00 / 0.96 / 0.57 | 0.44 / 0.61 / 0.31 |
| $T$ = 25% | 0.9960 | 1.0226 | 0.9985 | 1.0056 | 1.00 / 0.96 / 0.43 | 0.44 / 0.61 / 0.31 |
| $T$ = 30% | 0.9958 | 1.0219 | 0.9986 | 1.0053 | 1.00 / 0.94 / 0.43 | 0.44 / 0.61 / 0.31 |
| $T$ = 40% | 0.9964 | 1.0218 | 0.9987 | 1.0056 | 0.85 / 0.92 / 0.29 | 0.44 / 0.61 / 0.31 |
| $T$ = 50% | 0.9992 | 1.0211 | 0.9989 | 1.0063 | 0.04 / 0.90 / 0.29 | 0.44 / 0.61 / 0.31 |
| $T$ = 60% | 0.9985 | 1.0182 | 0.9990 | 1.0052 | 0.00 / 0.82 / 0.00 | 0.45 / 0.61 / 0.31 |
| $T$ = 70% | 0.9985 | 1.0027 | 0.9990 | 1.0001 | 0.00 / 0.00 / 0.00 | 0.45 / 0.61 / 0.31 |

**Table 18.** Mode 0 (formula from the hint) with 10 thresholds; 32 SRRIP leader sets, epoch 1024. “Fraction I-RRIP” is the share of ROI decisions that chose I-RRIP, “mean ratio” the average measured ratio.




(a) Speedup (b) Share of decisions that chose I-RRIP

**Figure 15.** Mode 0: sensitivity to the threshold $T$.

The measured ratio hardly changes within a trace: about 0.44 on Trace1, 0.61 on Trace2 and 0.31 on Trace3. Low thresholds ($T \leq 15\%$) almost always choose I-RRIP and match static I-RRIP; $T = 70\%$ never chooses it and equals SRRIP. In between, the traces switch to SRRIP in the order of their ratios: Trace3 first, then Trace1, then Trace2.

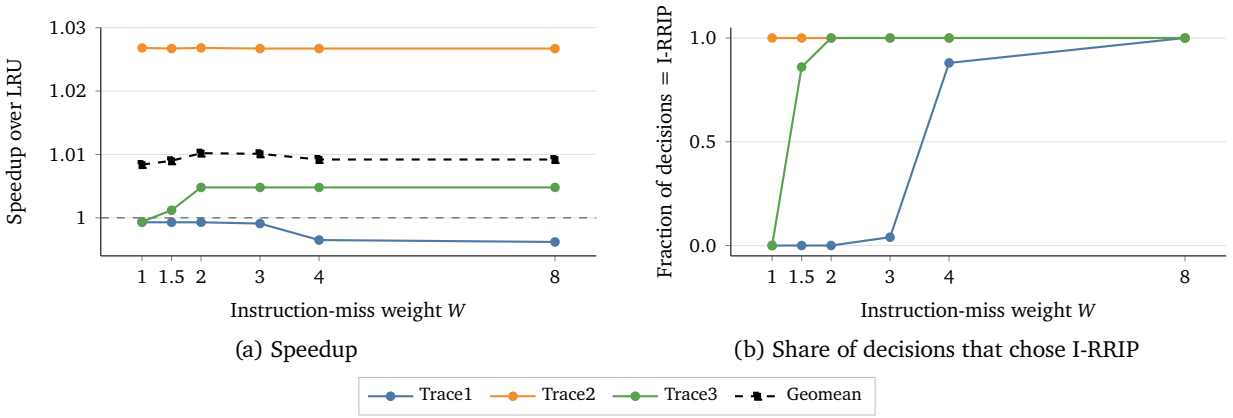
That order is the problem. **Trace3 gains from I-RRIP** (+0.50%) even though only about 31% of its leader traffic is instruction traffic, and **Trace1 loses** (−0.38%) even though 44% is. A threshold between the two ratios ($T =$ 30–40%) gives Trace3 SRRIP and Trace1 I-RRIP, which is exactly backwards. $T = 50\%$ gets Trace1 right but still takes I-RRIP away from Trace3. The best threshold is the lowest one ($T = 5\%$, +0.91%), so the formula works best when it does nothing.

The reason is that the ratio tells us *how much* of the L2 traffic is instruction traffic, not *whether* protecting it pays off. That depends on what the protection costs. On Trace1 the data has L2 reuse, and moving data insertion to RRPV 3 adds 2.57 data MPKI while removing only 1.30 instruction MPKI. To decide correctly, the policy has to measure the effect of I-RRIP itself, and that is what set dueling against I-RRIP leader sets does.

## 4.3 Miss duel: weight, leader sets and epoch

| Instr. miss weight | Speedup Trace1 | Speedup Trace2 | Speedup Trace3 | Geomean speedup | Fraction I-RRIP (T1/T2/T3) |
|---|---|---|---|---|---|
| $W = 1$ | 0.9993 | 1.0268 | 0.9994 | 1.0084 | 0.00 / 1.00 / 0.00 |
| $W = 1.5$ | 0.9993 | 1.0267 | 1.0012 | 1.0090 | 0.00 / 1.00 / 0.86 |
| $W = 2$ | 0.9993 | 1.0268 | 1.0048 | 1.0102 | 0.00 / 1.00 / 1.00 |
| $W = 3$ | 0.9991 | 1.0267 | 1.0048 | 1.0101 | 0.04 / 1.00 / 1.00 |
| $W = 4$ | 0.9965 | 1.0267 | 1.0048 | 1.0092 | 0.88 / 1.00 / 1.00 |
| $W = 8$ | 0.9962 | 1.0267 | 1.0048 | 1.0092 | 1.00 / 1.00 / 1.00 |

**Table 19.** Mode 1 with six instruction-miss weights $W$ (a data miss weighs 1; $W = 1.5$ is implemented as steps of 3 and 2), 32 + 32 leader sets, epoch 1024. Highlighted: submitted.




(a) Speedup (b) Share of decisions that chose I-RRIP

**Figure 16.** Mode 1: sensitivity to the instruction-miss weight $W$.

| Weight | 16 + 16 leader sets | 32 + 32 leader sets | 64 + 64 leader sets |
|---|---|---|---|
| $W = 1$ | 1.0104<br>0.00 / 1.00 / 1.00 ✓ | 1.0084<br>0.00 / 1.00 / 0.00 | 1.0095<br>1.00 / 1.00 / 1.00 |
| $W = 1.5$ | 1.0104<br>0.00 / 0.96 / 1.00 ✓ | 1.0090<br>0.00 / 1.00 / 0.86 ✓ | 1.0095<br>1.00 / 1.00 / 1.00 |
| $W = 2$ | 1.0093<br>0.93 / 1.00 / 1.00 | **1.0102**<br>0.00 / 1.00 / 1.00 ✓ | 1.0095<br>1.00 / 1.00 / 1.00 |

**Table 20.** Geometric-mean speedup for every combination of weight and number of leader sets, with the share of decisions that chose I-RRIP on Trace1 / Trace2 / Trace3 (✓: SRRIP on Trace1 and I-RRIP on Trace2 and Trace3, the best static choice per trace). Highlighted: submitted.

| Variant (mode 1, $W = 2$, 32 + 32 leaders unless stated) | Speedup Trace1 | Speedup Trace2 | Speedup Trace3 | Geomean speedup | Fraction I-RRIP (T1/T2/T3) |
|---|---|---|---|---|---|
| epoch 256 | 0.9986 | 1.0268 | 1.0048 | 1.0100 | 0.00 / 1.00 / 1.00 |
| **epoch 1024 (submitted)** | 0.9993 | 1.0268 | 1.0048 | 1.0102 | 0.00 / 1.00 / 1.00 |
| epoch 4096 | 0.9993 | 1.0267 | 1.0048 | 1.0102 | 0.00 / 1.00 / 1.00 |
| $W = 1$, epoch 256 | 0.9986 | 1.0268 | 0.9994 | 1.0082 | 0.00 / 1.00 / 0.00 |
| $W = 1$, epoch 1024 | 0.9993 | 1.0268 | 0.9994 | 1.0084 | 0.00 / 1.00 / 0.00 |
| $W = 1$, epoch 4096 | 0.9993 | 1.0268 | 0.9994 | 1.0084 | 0.00 / 1.00 / 0.00 |
| mode 0, $T = 25\%$, epoch 256 | 0.9957 | 1.0229 | 0.9986 | 1.0057 | 1.00 / 0.95 / 0.36 |
| mode 0, $T = 25\%$, epoch 1024 | 0.9960 | 1.0226 | 0.9985 | 1.0056 | 1.00 / 0.96 / 0.43 |
| mode 0, $T = 25\%$, epoch 4096 | 0.9962 | 1.0265 | 1.0048 | 1.0091 | 1.00 / 1.00 / 1.00 |

**Table 21.** Epoch length: the followers re-read the decision every *epoch* SRRIP-leader accesses.

**Weight.** With $W = 1$ (a plain miss count) and 32 + 32 leaders, the duel correctly picks SRRIP for Trace1 and I-RRIP for Trace2, but it also picks SRRIP for Trace3, where the two policies differ by only a few misses. $W = 2$ picks I-RRIP for Trace3 as well and gives +1.02%. $W = 3$ is a tie (+1.01%), while $W = 4$ and 8 give instruction misses too much weight and send Trace1 back to I-RRIP. $W = 2$ also has a physical meaning: an instruction miss stalls fetch, while the core can overlap part of a data miss with other work, so an instruction miss costs roughly twice as much.

**Leader sets and weight together (Table 20).** The leaders are only a sample of the cache, and on Trace1 and Trace3 the two policies are close. On Trace1, I-RRIP adds about 2.5 data MPKI and removes about 1.2 instruction MPKI, so with $W = 2$ the weighted misses of the two leader groups are almost equal. The winner then depends on which sets happen to be sampled, which is why neighbouring cells of the grid disagree on Trace1 or Trace3. Three settings choose the best policy on every trace (✓): 16 + 16 leaders with $W = 1$ or 1.5, and 32 + 32 leaders with $W = 2$. They are within 0.02% of each other, a tie. We submit **32 + 32 leaders with $W = 2$**: 32 leader sets per policy (6.25% of the 1024 sets together) is the size DIP uses for caches of this size, and it halves the sampling noise of 16 sets (the standard error of a sampled miss rate falls with the square root of the number of sets). 64 + 64 leaders run 12.5% of the cache with a fixed policy, and in our runs they also chose I-RRIP for Trace1. A wrong

choice is cheap on these traces anyway: the worst cell of the grid is +0.84%, still better than SRRIP (+0.02%) and close to static I-RRIP (+0.95%), because the duel is only unsure where the two policies are close.

**Epoch (Table 21).** PSEL is updated all the time; the epoch only sets how often the followers read it. Trace3 sends few requests to the L2, so with an epoch of 1024 it makes only 7 decisions in 25M instructions. That is still enough, because its preference never changes. For the duel, all three epochs give the same decisions (within 0.02%). For mode 0 the epoch changes which phases are sampled, but no epoch makes the formula choose the right policies.

## 4.4 Results

| Configuration | IPC | Speedup | L2C load MPKI | L2 instr. load MPKI | L2 data load MPKI |
|---|---|---|---|---|---|
| *Trace1* | | | | | |
| LRU (baseline) | 0.2321 | 1.0000 | 17.70 | 10.37 | 7.33 |
| SRRIP | 0.2320 | 0.9996 | 17.61 | 10.55 | 7.06 |
| I-RRIP | 0.2312 | 0.9962 | 18.88 | 9.25 | 9.63 |
| **Dyn-I-RRIP** | 0.2320 | 0.9993 | 17.68 | 10.54 | 7.14 |
| *Trace2* | | | | | |
| LRU (baseline) | 0.2499 | 1.0000 | 28.64 | 20.14 | 8.50 |
| SRRIP | 0.2504 | 1.0020 | 27.27 | 19.32 | 7.95 |
| I-RRIP | 0.2568 | 1.0276 | 23.25 | 12.33 | 10.92 |
| **Dyn-I-RRIP** | 0.2566 | 1.0268 | 23.33 | 12.53 | 10.79 |
| *Trace3* | | | | | |
| LRU (baseline) | 0.3820 | 1.0000 | 3.96 | 1.12 | 2.84 |
| SRRIP | 0.3816 | 0.9990 | 4.04 | 1.20 | 2.85 |
| I-RRIP | 0.3839 | 1.0050 | 3.90 | 0.73 | 3.17 |
| **Dyn-I-RRIP** | 0.3838 | 1.0048 | 3.91 | 0.75 | 3.16 |
| *Average (geometric mean for IPC and speedup, arithmetic mean otherwise)* | | | | | |
| LRU (baseline) | 0.2809 | 1.0000 | 16.77 | 10.54 | 6.22 |
| SRRIP | 0.2809 | 1.0002 | 16.31 | 10.36 | 5.95 |
| I-RRIP | 0.2836 | 1.0095 | 15.34 | 7.44 | 7.91 |
| **Dyn-I-RRIP** | 0.2838 | 1.0102 | 14.97 | 7.94 | 7.03 |

**Table 22.** Task 2B: IPC, speedup over LRU and L2 load MPKIs (no prefetcher). Dyn-I-RRIP = mode 1, $W = 2$, 32 + 32 leaders, epoch 1024.

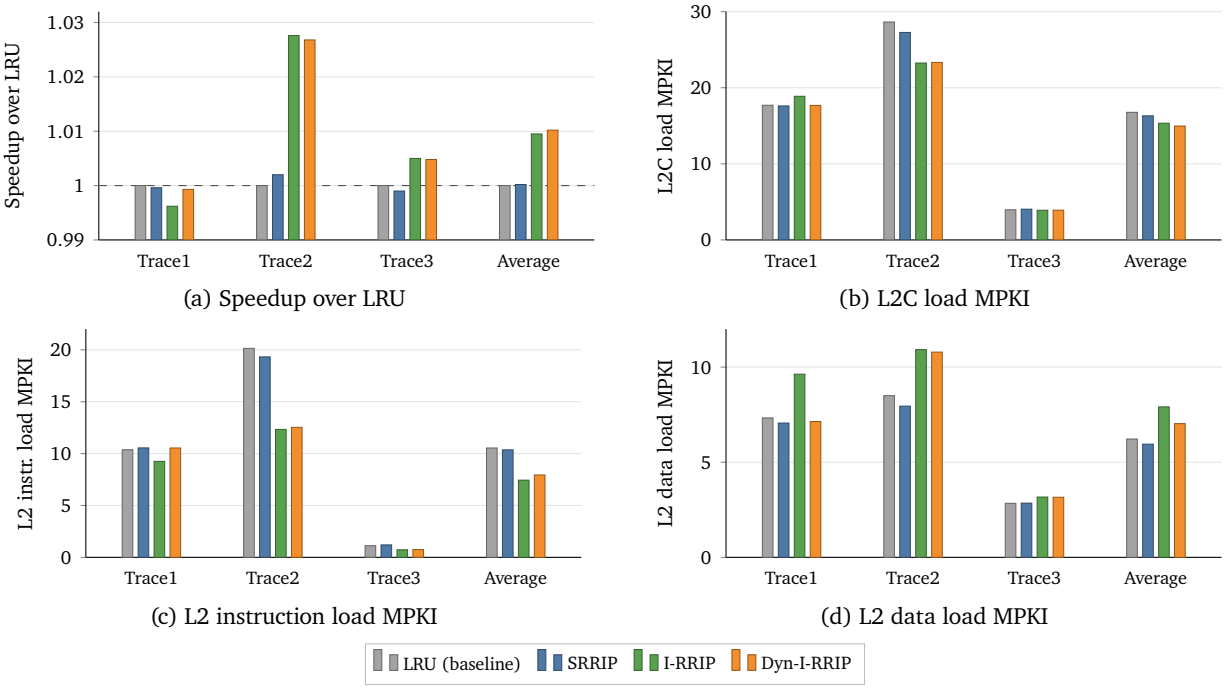



**Figure 17.** Task 2B: speedup and L2C / L2 instruction / L2 data load MPKI.

Dynamic I-RRIP keeps the gains of I-RRIP where they exist (Trace2 +2.68%, Trace3 +0.48%) and falls back to SRRIP on Trace1 (−0.07% instead of −0.38% with static I-RRIP). As a result it beats both SRRIP and I-RRIP in geometric mean (+1.02% vs +0.02% and +0.95%), and Trace1's data load MPKI goes back to the SRRIP level (7.14).

**Storage.** On top of SRRIP (2-bit RRPV per line, 16384 bits), the policy needs the instruction bit per line (8192 bits), a 10-bit PSEL, a 10-bit epoch counter and the 1-bit follower mode: 8213 bits ≈ 1.0 KiB in total. Leader sets are recognised by comparing two 5-bit fields of the set index, so no table is needed. (Mode 0 would need four 16-bit counters, 64 bits, instead of PSEL.)

# 5 Task 3: SPP with Prefetch-Aware Dynamic I-RRIP

## 5.1 What the L2 sees from SPP

SPP (Task 1) sits at the L1D. Each prefetch first looks up the L1D. If it misses there, it goes to the L2 as a `PREFETCH` request, and if it misses in the L2 too, the line is filled into the L2 on its way to the L1D. (SPP fills the L1D when fewer than half of the L1D MSHRs are busy; otherwise it fills only the L2.) So the replacement policy sees three new events:

- a prefetch fill,
- a prefetch request that hits a line already in the L2, and
- later, perhaps, a demand hit on a line that a prefetch brought in (`BLOCK::prefetch` is still set; the cache clears it right after calling the policy).

Counting these events shows that only 2.3% / 0.8% / 0.9% of the prefetched lines filled into the L2 ever get a demand hit at the L2 (columns *pf fills* and *first use* in Table 23, counted over warm-up and simulation). Useful prefetches are consumed in the L1D. The L2 copy is a duplicate that the program will not ask the L2 for again unless the L1D loses it first.

## 5.2 Prefetch-aware design

The prefetch handling is part of `replacement/dyn_irrip.l2c_repl` (`DYN_PF_AWARE`, on by default). It applies to data prefetches in all sets and in both follower modes:

1. **Insertion:** a prefetch fill is inserted at RRPV 3 (distant). A prefetch is a guess, not a demand, and it will most likely be used in the L1D. In I-RRIP mode data gets RRPV 3 anyway; in SRRIP mode (Trace1) this stops prefetched lines from pushing out demand data inserted at 2.
2. **No promotion on a prefetch hit:** a prefetch request that hits in the L2 is about to copy the line into the L1D, which says nothing about future L2 reuse. SRRIP and I-RRIP would move the line to RRPV 0.
3. **First demand use:** the first demand hit on a prefetched line proves the prefetch right, so the line is promoted to RRPV 0 like any reused line.
4. **Eviction:** among the RRPV-3 candidates, an unused prefetched data line is evicted first, then other data, then instructions.
5. **Set dueling sees demand traffic only:** prefetch misses are not counted in PSEL (or in the mode-0 ratio). Prefetch traffic depends on the prefetcher, not on how well a replacement policy keeps instruction or data lines. Counting it would add data misses to both leader groups and dilute the instruction-miss signal the duel relies on.

None of this needs new state: `BLOCK::prefetch` already exists in every cache line. With no prefetcher none of these paths runs, so the Task 2B results are exactly the results of this file.

## 5.3 Choosing the L2 policy and the prefetch rules

| Configuration (SPP at L1D) | Speedup Trace1 | Speedup Trace2 | Speedup Trace3 | Geomean speedup | L1D acc. (%) | L2 instr. MPKI | L2 data MPKI | pf fills / first use |
|---|---|---|---|---|---|---|---|---|
| SPP + LRU | 1.0083 | 1.0135 | 1.0126 | 1.0115 | 54.0 | 10.58 | 5.59 | – |
| SPP + SRRIP | 1.0081 | 1.0155 | 1.0120 | 1.0118 | 54.2 | 10.39 | 5.32 | – |
| SPP + I-RRIP | 1.0055 | 1.0412 | 1.0177 | 1.0214 | 53.8 | 7.46 | 7.18 | – |
| SPP + Dyn-I-RRIP (pf-unaware) | 1.0080 | 1.0404 | 1.0176 | 1.0219 | 54.0 | 7.97 | 6.38 | – |
| PF-aware: insert pf at 2 | 1.0079 | 1.0365 | 1.0146 | 1.0196 | 54.2 | 8.08 | 6.31 | 230709 / 8087 |
| PF-aware: first use to 1 | 1.0079 | 1.0405 | 1.0181 | 1.0221 | 54.0 | 7.95 | 6.37 | 241248 / 3308 |
| PF-aware: first use to 2 | 1.0079 | 1.0405 | 1.0180 | 1.0220 | 54.0 | 7.94 | 6.38 | 240974 / 3266 |
| PF-aware: no pf-first evict | 1.0077 | 1.0405 | 1.0183 | 1.0221 | 54.1 | 7.94 | 6.38 | 240007 / 3430 |
| PF-aware: pf in PSEL | 1.0077 | 1.0405 | 1.0125 | 1.0201 | 54.2 | 8.05 | 6.31 | 237208 / 3491 |
| **SPP + Dyn-I-RRIP-PF** | 1.0079 | 1.0406 | 1.0180 | 1.0221 | 54.1 | 7.94 | 6.38 | 241044 / 3357 |

**Table 23.** SPP combined with each L2 policy, and the prefetch-aware variants (one rule changed at a time). Speedup is over the baseline without prefetcher and with LRU. Accuracy and MPKI are means over the traces. pf fills / first use: prefetch fills into the L2 and first demand uses of such lines, summed over the traces. Highlighted: submitted.

**Policy.** With SPP added, the L2 policies keep their Task 2 order: SRRIP +1.18%, I-RRIP +2.14% and prefetch-unaware Dyn-I-RRIP +2.19% (SPP + LRU: +1.15%). The instruction-miss savings of I-RRIP add on top of the data-miss savings of SPP. Dyn-I-RRIP beats static I-RRIP with SPP as well, again by leaving Trace1 on SRRIP (Trace1: 1.0080 vs 1.0055).

**Prefetch rules, one at a time.** The complete prefetch-aware policy (+2.21%) and the prefetch-unaware one (+2.19%) are a tie. Since I-RRIP already inserts every data line at RRPV 3, prefetched or not, most prefetch rules repeat what the unaware policy does in I-RRIP mode anyway. Removing single rules from the aware policy shows which ones carry weight:

- **Distant insertion matters.** Inserting prefetches at RRPV 2 instead of 3 costs 0.24%. The prefetched lines then outlive other data lines and push out instruction lines: the L2 instruction MPKI rises on Trace2 (12.59 → 12.90) and Trace3 (0.75 → 0.85).
- **Keeping prefetches out of the duel matters.** Counting prefetch misses in PSEL costs 0.19%. These extra misses are data misses that both leader groups suffer alike, so they add noise but no signal. On Trace3, which has few L2 demand misses, they drag PSEL towards SRRIP (I-RRIP chosen in 0.43 of the decisions instead of 1.00).
- **First-use promotion and prefetch-first eviction are ties** (promotion to RRPV 1: +2.21%, to RRPV 2: +2.20%, no prefetch-first eviction: +2.21%, vs +2.21%). Only 2.3% / 0.8% / 0.9% of the prefetched lines are ever used at the L2, so the first-use rule rarely runs. And with data inserted at RRPV 3, the RRPV-3 candidates are nearly always prefetched or other dead data lines anyway. We keep both rules because they are free and follow the same reasoning, but the data does not show that they help.

## 5.4 Results

| Configuration | IPC | Speedup | L1D accuracy (%) | L1D coverage (%) | L1D load MPKI | L2C load MPKI | L2 instr. MPKI | L2 data MPKI |
|---|---|---|---|---|---|---|---|---|
| *Trace1* | | | | | | | | |
| Baseline (no pf, LRU) | 0.2321 | 1.0000 | – | – | 10.68 | 17.70 | 10.37 | 7.33 |
| SPP + LRU | 0.2341 | 1.0083 | 57.6 | 9.42 | 9.67 | 16.91 | 10.42 | 6.50 |
| Dyn-I-RRIP (no pf) | 0.2320 | 0.9993 | – | – | 10.68 | 17.68 | 10.54 | 7.14 |
| SPP + Dyn-I-RRIP | 0.2340 | 1.0080 | 57.9 | 9.47 | 9.67 | 16.86 | 10.55 | 6.31 |
| **SPP + Dyn-I-RRIP-PF** | 0.2340 | 1.0079 | 57.9 | 9.46 | 9.67 | 16.76 | 10.49 | 6.27 |
| *Trace2* | | | | | | | | |
| Baseline (no pf, LRU) | 0.2499 | 1.0000 | – | – | 13.54 | 28.64 | 20.14 | 8.50 |
| SPP + LRU | 0.2533 | 1.0135 | 63.2 | 4.46 | 12.93 | 28.10 | 20.19 | 7.91 |
| Dyn-I-RRIP (no pf) | 0.2566 | 1.0268 | – | – | 13.55 | 23.33 | 12.53 | 10.79 |
| SPP + Dyn-I-RRIP | 0.2600 | 1.0404 | 63.6 | 4.37 | 12.94 | 22.78 | 12.60 | 10.18 |
| **SPP + Dyn-I-RRIP-PF** | 0.2601 | 1.0406 | 63.6 | 4.39 | 12.94 | 22.78 | 12.59 | 10.19 |
| *Trace3* | | | | | | | | |
| Baseline (no pf, LRU) | 0.3820 | 1.0000 | – | – | 3.40 | 3.96 | 1.12 | 2.84 |
| SPP + LRU | 0.3868 | 1.0126 | 41.0 | 10.44 | 3.05 | 3.50 | 1.13 | 2.37 |
| Dyn-I-RRIP (no pf) | 0.3838 | 1.0048 | – | – | 3.40 | 3.91 | 0.75 | 3.16 |
| SPP + Dyn-I-RRIP | 0.3887 | 1.0176 | 40.6 | 10.50 | 3.05 | 3.42 | 0.76 | 2.66 |
| **SPP + Dyn-I-RRIP-PF** | 0.3889 | 1.0180 | 40.8 | 10.58 | 3.04 | 3.41 | 0.75 | 2.66 |
| *Average (geometric mean for IPC and speedup, arithmetic mean otherwise)* | | | | | | | | |
| Baseline (no pf, LRU) | 0.2809 | 1.0000 | – | – | 9.21 | 16.77 | 10.54 | 6.22 |
| SPP + LRU | 0.2841 | 1.0115 | 54.0 | 8.10 | 8.55 | 16.17 | 10.58 | 5.59 |
| Dyn-I-RRIP (no pf) | 0.2838 | 1.0102 | – | – | 9.21 | 14.97 | 7.94 | 7.03 |
| SPP + Dyn-I-RRIP | 0.2870 | 1.0219 | 54.0 | 8.11 | 8.55 | 14.35 | 7.97 | 6.38 |
| **SPP + Dyn-I-RRIP-PF** | 0.2871 | 1.0221 | 54.1 | 8.15 | 8.55 | 14.32 | 7.94 | 6.38 |

**Table 24.** Task 3: IPC, speedup over the baseline (no prefetcher, LRU), L1D prefetch accuracy and coverage, and load MPKI of the L1D, L2C, L2 instructions and L2 data. Accuracy and coverage are undefined without a prefetcher (–).

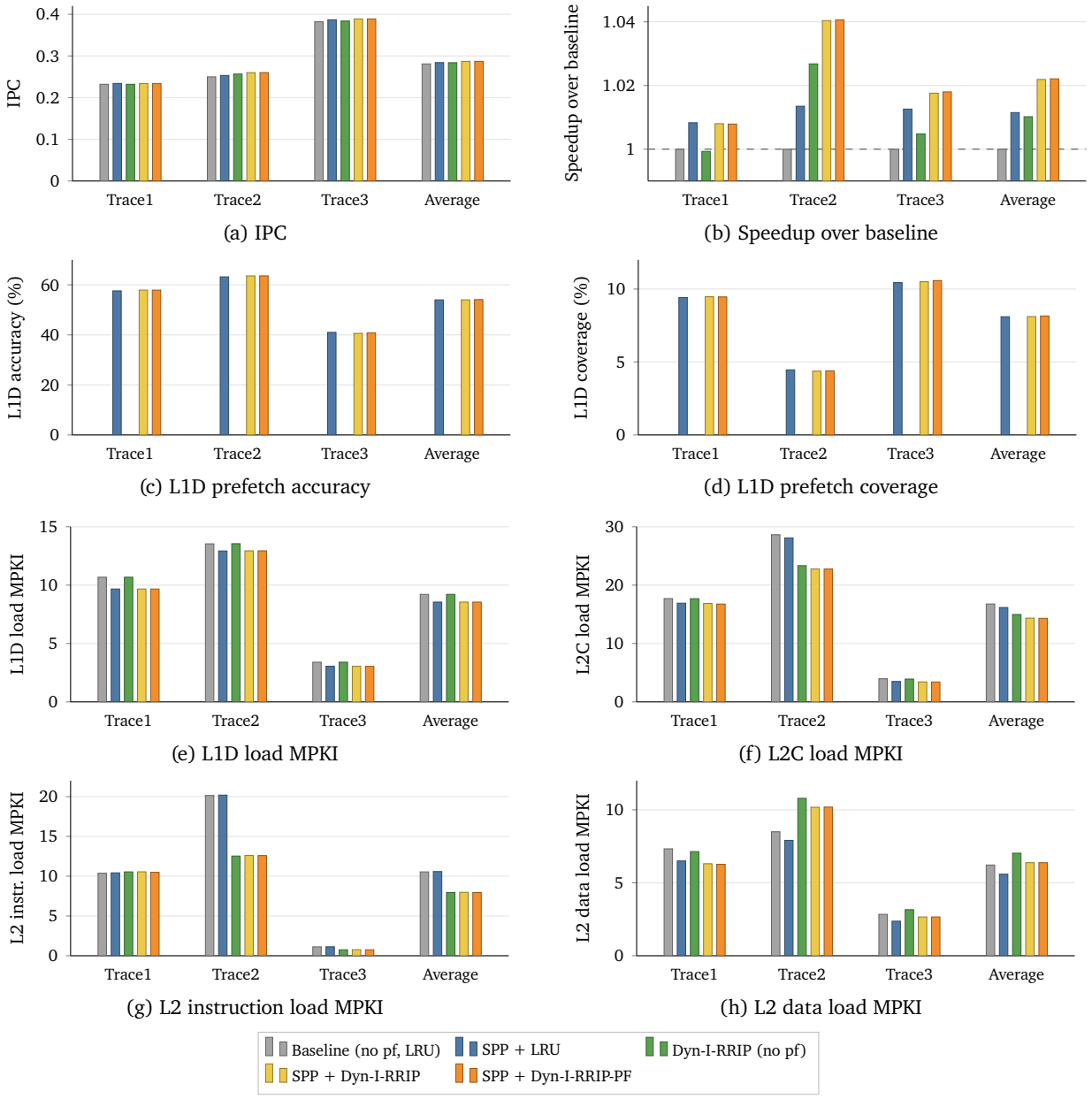



**Figure 18.** Task 3: every metric of Table 24. The last group is the average (geometric mean for IPC and speedup). Configurations without a prefetcher have no accuracy or coverage bars.

**Outcome.** SPP + prefetch-aware Dyn-I-RRIP reaches a geometric-mean speedup of **+2.21%**, against +1.15% for SPP alone (with LRU) and +1.02% for Dyn-I-RRIP without prefetching, so it beats both. Per trace:

- Trace1: 1.0079 vs 1.0083 (SPP) and 0.9993 (Dyn-I-RRIP);
- Trace2: 1.0406 vs 1.0135 (SPP) and 1.0268 (Dyn-I-RRIP);
- Trace3: 1.0180 vs 1.0126 (SPP) and 1.0048 (Dyn-I-RRIP).

The two mechanisms remove different misses, so their gains add up. SPP removes L1D data misses (coverage 8.1%, L1D load MPKI 9.21 → 8.55), and Dyn-I-RRIP removes L2 instruction misses (mean 10.58 → 7.94), which SPP cannot touch. On Trace1, where the duel keeps SRRIP, the result stays within 0.04% of SPP + LRU: the L2 policy has almost no effect on that trace (LRU and SRRIP differ by 0.04% there as well).

**SPP accuracy and coverage** stay the same with every L2 policy (accuracy 54.0% vs 54.1%, coverage 8.10% vs 8.15% for SPP + LRU vs the submitted combination). The L2 policy does not change what SPP predicts, only how long the L2 keeps the lines.